

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2015

Huấn luyện viên: Yu Linyun

China Computer Federation, 2015

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015

I0I China candidate team papers / National training team 2015 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2015. Tập được tạo bằng pdfTeX nhưng lớp văn bản không ánh xạ đúng về Unicode: ngay cả mục lục khi trích xuất bằng pdftotext cũng trở thành mojibake. Bản dịch này chuyển ngữ phân đầu sách, mục lục đọc được từ các trang đã render, và ba bài đầu tiên của tập sách, được OCR từ ảnh trang.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015*. Huấn luyện viên ghi trên bìa là Yu Linyun. Đơn vị xuất bản là China Computer Federation.

Ghi chú trích xuất. Do lớp văn bản của PDF nguồn không dùng được, các bài đã dịch được đọc từ ảnh render bằng pdftoppm và OCR bằng tesseract -l chi_sim+eng, sau đó đối chiếu trực quan ở các trang biên. Hiện bản dịch đã hoàn thành hai bài đầu tiên: bài “Mở rộng suffix automaton trên trie” dùng các trang PDF vật lý 5–20, tương ứng trang in 1–16; bài “Bàn về ứng dụng tư tưởng heuristic trong thi tin học” dùng các trang PDF vật lý 21–40, tương ứng trang in 17–36; bài “Bàn về một số phương pháp khớp chuỗi” dùng các trang PDF vật lý 41–69, tương ứng trang in 37–65. Trang PDF vật lý 70, trang in 66, là trang trắng; trang PDF vật lý 71, trang in 67, là đầu bài kế tiếp về suffix automaton.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Mở rộng suffix automaton trên trie	Liu Yanyi, Trường Yali, Trường Sa, Hồ Nam
17	Bàn về ứng dụng tư tưởng heuristic trong thi tin học	Ren Zhizhou, Trường Trung học số 1 Thiệu Hưng
37	Bàn về một số phương pháp khớp chuỗi	Wang Jianhao, Trường Trung học số 1 Thiệu Hưng
67	Suffix automaton và ứng dụng	Zhang Tianyang, Trường Trung học số 1 Trường Sa, Hồ Nam
81	Phép toán trên hàm sinh và bài toán đếm tổ hợp	Jin Ce, Trường Học Quân Hàng Châu, Chiết Giang
103	Báo cáo ra đề <i>yc's bonus</i>	Liu Jiancheng, Trường Yali, Trường Sa
113	Bàn về ứng dụng chia khối trong một lớp bài toán trực tuyến	Zou Daoyao, Trường Trung học Trần Hải, Ninh Ba
129	Thuật toán liên quan tới cactus và ứng dụng	Wang Yisong, Trường Học Quân Hàng Châu, Chiết Giang
149	Bàn về thuật toán matching trên đồ thị và ứng dụng	Chen Haobo, Trường Trường Quận, Trường Sa, Hồ Nam
175	Bàn về các bài toán vật lý trong thi tin học	Chen Siyu, Trường Trung học số 2 Hàng Châu
199	Báo cáo ra đề <i>Đề bài bị thất lạc</i>	Yu Jiping, Trường Trung học số 24 Đại Liên
205	Một số kỹ thuật tối ưu DP	Zhang Hengjie, Trường Trung học số 1 Thiệu Hưng
217	Báo cáo ra đề <i>Product</i>	Du Yuhao, Trường Trung học Trần Hải, Ninh Ba
231	Bước đầu tìm hiểu một lớp bài toán lấy mã nguồn làm đầu vào	Lu Xiaochen, Học viện Giao Xuyên Trần Hải, Ninh Ba
271	Tính chất, ứng dụng và thuật toán nhanh cho lũy thừa tập hợp	Lu Kaifeng, Trường Trung học số 2 Vũ Hán, Hồ Bắc

3 Mở rộng suffix automaton trên trie

Liu Yanyi

Trường Yali, Trường Sa, Hồ Nam

Tóm tắt

Bài viết phân tích chi tiết một automaton hữu hạn xác định có thể nhận ra mọi hậu tố của một trie. Tác giả đưa ra và chứng minh các kết luận rằng số trạng thái là tuyến tính, tổng số hàm chuyển không vượt quá số nút của trie nhân với kích thước bảng chữ cái, đồng thời trình bày một thuật toán xây dựng khả thi, có độ phức tạp tuyến tính khi được phép xử lý offline. Bài viết cũng phân tích độ phức tạp của thuật toán xây dựng online, giới thiệu một số ứng dụng liên quan của suffix automaton, với hy vọng gợi mở thêm

các hướng sử dụng cấu trúc này.

3.1 Lời nói đầu

Trong thi thuật toán, xử lý xâu là một mảng rất quan trọng. Nhìn chung có hai loại bài toán lớn: bài toán có mẫu cố định và bài toán có văn bản chính cố định. Khi mẫu cố định, có thể có nhiều văn bản chính khác nhau. Nếu chỉ có một mẫu, thuật toán KMP nổi tiếng [2, 3] xử lý được lớp bài toán này; cũng có thể dùng hashing xâu [13] để đếm số lần xuất hiện của mẫu. Nếu có nhiều mẫu, thuật toán AC automaton quen thuộc [1] đảm nhiệm tốt lớp bài toán đó. Nếu văn bản chính cố định, ta thường xây suffix array [11], suffix tree [4, 5] hoặc suffix automaton [7] cho văn bản chính. Lớp bài toán này đã được nghiên cứu đầy đủ trong cộng đồng thi thuật toán. Chẳng hạn, Chen Lijie đã nêu nhiều ứng dụng của suffix automaton trên một xâu trong bài giảng giao lưu trại đông [8], còn Luo Suijian giải thích chi tiết việc dùng suffix array trong thi tin học trong luận văn đội tuyển quốc gia của mình [12]. Khi đề bài cho phép xử lý offline, hai loại bài toán trên thường có thể chuyển đổi cho nhau, thể hiện tính linh hoạt của các thuật toán xử lý xâu.

Trong kỳ chọn đội tuyển tỉnh Chiết Giang năm 2015 xuất hiện bài toán sau: cho một cây có $n \leq 10^5$ nút, số lá không quá 20, mỗi cạnh mang một ký tự. Đường đi giữa hai nút trên cây tạo thành một xâu; hỏi có tất cả bao nhiêu xâu con khác nhau.¹ Bài này rõ ràng thuộc loại văn bản chính cố định, nhưng văn bản chính không còn là một xâu mà là một cây. Nếu trực tiếp liệt kê mọi cặp đỉnh rồi dùng trie [1, 13] để kiểm tra hai xâu có bằng nhau hay không, độ phức tạp là $O(n^3)$. Ngay cả dùng hashing xâu [13], thời gian vẫn là $O(n^2)$, không thể qua được dữ liệu.

Ta chuyển hóa bài toán như sau. Vì chỉ có 20 lá, với mỗi lá làm gốc, dựng một trie riêng rồi hợp nhất tất cả các trie này thành một trie mới. Khi đó mọi xâu xuất hiện trên cây ban đầu chắc chắn xuất hiện trong trie mới; văn bản chính đã trở thành một trie.

Để giải quyết vấn đề trên, bài viết phân tích automaton hữu hạn xác định nhận ra toàn bộ hậu tố của một trie và đề xuất một cách xây dựng. Nội dung chính gồm năm phần:

1. đưa một số khái niệm trên xâu sang trie, đồng thời giới thiệu suffix automaton của trie;
2. phân tích suffix link và xây dựng cấu trúc cây parent;
3. chứng minh số trạng thái là tuyến tính và nêu chặn trên cho tổng số hàm chuyển trong automaton;
4. mô tả thuật toán xây suffix automaton cho trie; khi được phép offline, độ phức tạp của nó là tuyến tính theo kích thước automaton;
5. trình bày một số ứng dụng của suffix automaton trên trie, qua đó làm rõ hơn cách nhìn về suffix automaton.

3.2 Suffix automaton của trie

Trong bài viết này, mọi ký tự của xâu và mọi ký tự trên cạnh của trie đều thuộc một bảng chữ cái hữu hạn A . Tập mọi xâu sinh bởi bảng chữ cái này là A^* ; xâu rỗng được ký hiệu là ε . Tập không chứa xâu rỗng là $A^+ = A^* \setminus \{\varepsilon\}$. Dùng $|s|$ để chỉ độ dài xâu s , s_i là ký tự thứ i , và $s_{i,j}$ là xâu gồm các ký tự từ vị trí i đến vị trí j của s . Ký hiệu $a \cdot b$ là xâu thu được khi nối a với b ; dấu nhân có thể được lược bỏ.

Tập mọi hậu tố của một xâu s là

$$S(s) = \{s_{i,|s|} \mid 1 \leq i \leq |s|\}.$$

Với một tập xâu S , $\text{Minl}(S)$ và $\text{Maxl}(S)$ lần lượt là độ dài của xâu ngắn nhất và dài nhất trong S .

¹ZJOI 2015 Day 1, *Vùng đất huyền tưởng được chut thần ưu ái*, tác giả Chen Lijie.

Dùng T để chỉ một trie, và dùng $T_x, T_y, T_z, \dots$ để chỉ các nút của T . Xây nhận được bằng cách nối theo thứ tự các ký tự trên đường đi từ nút T_x đến một nút T_y trong cây con của nó được ký hiệu là $T_{x,y}$. Nếu không nói rõ, khi dùng $T_{x,y}$, ta luôn ngầm hiểu T_y nằm trong cây con của T_x .

Một tiền tố của T là

$$P_{T_x} = T_{\text{root},x} \quad (\text{root là gốc của } T, T_x \in T).$$

Tập mọi xâu con của T là

$$F(T) = \{T_{x,y} \mid T_x, T_y \in T, T_y \text{ nằm trong cây con của } T_x\}.$$

Tương tự, tập mọi hậu tố của T là

$$S(T) = \{T_{x,y} \mid T_x, T_y \in T, T_y \text{ là nút lá}\}.$$

Xây suffix automaton cho trie nghĩa là xây một automaton hữu hạn có thể nhận ra mọi xâu trong $S(T)$, đồng thời cố gắng giảm số trạng thái và số chuyển xuống càng nhỏ càng tốt. Tương tự suffix automaton trên một xâu, ta định nghĩa vị trí xuất hiện của một xâu trên trie:

$$\text{Right}(s) = \{T_y \mid T_y \in T, \exists T_x \in T, T_y \text{ nằm trong cây con của } T_x, T_{x,y} = s\}.$$

Định nghĩa quan hệ tương đương R_T ; khi không gây nhầm lẫn viết là R :

$$aRb \iff \text{Right}(a) = \text{Right}(b) \quad (a, b \in A^*).$$

Gọi $R(s)$ là tập mọi xâu tương đương với s , tức một lớp tương đương. Trong automaton, mỗi trạng thái tương ứng với một lớp tương đương.

Gọi suffix automaton xây từ T là $\Phi(T)$, trạng thái đầu là $\Phi(T)_\varepsilon$. Ký hiệu $\Phi(T)_s$, với $s \in F(T)$, là trạng thái sau khi đọc s . Hàm chuyển được định nghĩa bởi chuyển nhân a từ $\Phi(T)_s$ tới $\Phi(T)_{sa}$ khi $a \in A$ và $sa \in F(T)$. Khi nối hai xâu s_1, s_2 mà $s_1, s_2, s_1s_2 \in F(T)$, ta cũng có $\Phi(T)_{s_1}$ đọc s_2 tới $\Phi(T)_{s_1s_2}$. Theo định nghĩa,

$$\Phi(T)_a = \Phi(T)_b \iff aRb \quad (a, b \in F(T)).$$

Nếu a là một xâu tương ứng với trạng thái $\Phi(T)_b$, thì tập xâu tương ứng với trạng thái $\Phi(T)_b$ chính là $R(b)$.

Hình a. Trong trie minh họa của bản gốc, b tương đương với ab ; các nút màu đỏ là tập Right của chúng.

Để phân tích số trạng thái của toàn bộ cấu trúc, tức số lớp tương đương R trong $F(T)$, ta cần xét một số tính chất của các trạng thái suffix automaton. Mục tiêu là làm số trạng thái nhỏ nhất có thể.

Bổ đề 2.1. Với $x, y, z, u \in F(T)$, ta có:

- (i) $xRy \Rightarrow (x \in S(y) \text{ hoặc } y \in S(x))$,
- (ii) $x \in S(z) \Rightarrow \text{Right}(z) \subseteq \text{Right}(x)$,
- (iii) $(x \in S(z), z \in S(u), xRu) \Rightarrow zRuRx$.

Chứng minh. (i) Không mất tính tổng quát, giả sử $|x| \leq |y|$ và $Right(x) = Right(y)$. Lấy tùy ý $T_v \in Right(x)$. Khi đó $x \in S(P_{T_v})$ và $y \in S(P_{T_v})$, nên $x \in S(y)$.

(ii) Với mọi $T_v \in Right(z)$, ta có $z \in S(P_{T_v})$. Vì $x \in S(z)$, suy ra $x \in S(P_{T_v})$, do đó $T_v \in Right(x)$. Vậy $Right(z) \subseteq Right(x)$.

(iii) Từ (ii), $z \in S(u)$ cho $Right(u) \subseteq Right(z)$, và $x \in S(z)$ cho $Right(z) \subseteq Right(x)$. Lại có xRu , nên $Right(z) = Right(u) = Right(x)$, tức $zRuRx$.

Mệnh đề (i) cũng cho thấy trong cùng một lớp tương đương R , không thể có hai xâu khác nhau nhưng cùng độ dài.

Hệ quả 2.2. Với một lớp tương đương $R(a)$, gọi x là xâu dài nhất trong $R(a)$, và y là xâu ngắn nhất trong $R(a)$. Khi đó

$$R(a) = \{z \mid z \in S(x) \text{ và } |y| \leq |z| \leq |x|\}.$$

Điều này suy ra trực tiếp từ Bổ đề 2.1.

3.3 Suffix link

Trong thuật toán xây suffix automaton cho một xâu, suffix link giữ vai trò then chốt [7, 6], và toàn bộ thuật toán rất giống KMP [3]. Khi xây suffix automaton cho trie, ta cũng giữ suffix link và so sánh quá trình xây dựng với thuật toán xây automaton tối thiểu xác định nhận ra tập $A^*\{P_{T_x}\}$, tức cách xây AC automaton quen thuộc [1]. Tương tự hàm fail trong AC automaton, ta định nghĩa suffix link của suffix automaton trên trie như sau.

Suffix link là một hàm $s_T : A^* \setminus \{\varepsilon\} \rightarrow A^*$, khi không nhầm lẫn viết là s :

$$s_T(x) = \{z \mid z \in S(x), |z| = \text{Minl}(R_T(x)) - 1\}.$$

Nói cách khác, đó là hậu tố dài nhất y của x sao cho xR_Ty không đúng.

Bổ đề 3.1. Với $x, y \in F(T)$, ta có:

- (i) $xRy \Rightarrow s(x) = s(y)$,
- (ii) $Right(x) \subset Right(s(x))$,
- (iii) $\text{Minl}(R(x)) - 1 = |s(x)| = \text{Maxl}(R(s(x)))$.

Chứng minh. (i) Không mất tính tổng quát, giả sử $|x| \leq |y|$. Theo định nghĩa, $|s(x)| = \text{Minl}(R(x)) - 1$, $|s(y)| = \text{Minl}(R(y)) - 1$, mà $R(x) = R(y)$, nên $|s(x)| = |s(y)| < |x| \leq |y|$. Theo Bổ đề 2.1(i), $x \in S(y)$. Lại có $s(x) \in S(x)$ và $s(y) \in S(y)$, do đó $s(x) = s(y)$.

(ii) Theo Bổ đề 2.1(ii), $Right(x) \subseteq Right(s(x))$. Vì $xRs(x)$ là sai, hai tập này không bằng nhau, nên bao hàm là nghiêm ngặt.

(iii) Vì $s(x) \in R(s(x))$, ta có $|s(x)| \leq \text{Maxl}(R(s(x)))$. Giả sử tồn tại xâu y tương đương với $s(x)$ và dài nhất trong lớp đó. Khi ấy $Right(y) = Right(s(x))$, đồng thời $s(x) \in S(y)$. Dùng phản chứng: nếu $|y| > |s(x)|$, lấy $T_v \in Right(x)$. Từ (ii), $T_v \in Right(s(x))$, suy ra $T_v \in Right(y)$, nên $y \in S(P_{T_v})$ và $x \in S(P_{T_v})$. Nếu $|y| > |x|$ thì $x \in S(y)$, theo Bổ đề 2.1(ii) sẽ có $Right(s(x)) = Right(y) \subseteq Right(x)$, mâu thuẫn với (ii). Nếu $|y| \leq |x|$, vì xRy không đúng nên $s(x) = y$, mâu thuẫn với $|y| > |s(x)|$. Vậy mệnh đề được chứng minh.

Từ (i), mọi xâu trong cùng một lớp tương đương $R(a)$ có suffix link trở tới cùng một xâu. Vì vậy ta có thể gán suffix link cho từng trạng thái của suffix automaton. Cụ thể, suffix link của $R(a)$ trở tới $R(s(a))$.

Theo (ii) hoặc (iii), các suffix link này tạo thành một cây; ta gọi nó là **cây parent**. Định nghĩa $Fa(\Phi(T)_s)$ là suffix link của trạng thái $\Phi(T)_s$, tức

$$Fa(\Phi(T)_a) = \Phi(T)_b \iff s(a)Rb.$$

Bổ đề 3.2. Với $x \in F(T)$, lấy tùy ý $T_v \in \text{Right}(x)$. Nếu $|P_{T_v}| > \text{Maxl}(R(x))$ và đặt $a = P_{T_v}$, thì

$$s(a_{|a|-\text{Maxl}(R(x)), |a|})Rx.$$

Chứng minh. Đặt $b = a_{|a|-\text{Maxl}(R(x)), |a|}$. Từ giả thiết có $x \in S(b)$. Theo Hệ quả 2.2, vì $|b| > \text{Maxl}(R(x))$, nên bRx không đúng, còn hậu tố độ dài $\text{Maxl}(R(x))$ của b lại tương đương với x . Vì vậy b không tương đương với hậu tố đó; theo định nghĩa suffix link, $s(b)$ chính là hậu tố này, và $s(b)Rx$.

Hình b. Cây trong bản gốc là cây parent của suffix automaton được xây từ trie ở Hình a.

3.4 Số trạng thái tuyến tính

Với suffix automaton xây cho một xâu x , kích thước automaton là $O(|x|)$ [7, 6]. Ta cũng mong suffix automaton xây cho trie có quy mô trạng thái tuyến tính. Sau khi định nghĩa suffix link, cần phân tích kỹ hơn $\Phi(T)$.

Bổ đề 4.1. Với $x, y \in F(T)$, ta có:

- (i) $\forall y, \neg(xRs(y)) \Rightarrow |\text{Right}(x)| = 1$,
- (ii) $|\text{Right}(x)| = 1 \Rightarrow P_{T_v}Rx \quad (\text{Right}(x) = \{T_v\})$,
- (iii) $\forall T_v \in \text{Right}(x), \neg(xRP_{T_v})$
 $\Rightarrow \exists u, v, s(u)Rs(v)Rx \text{ và } \neg(uRv).$

Chứng minh. (i) Chứng minh bằng phản chứng. Giả sử $|\text{Right}(x)| > 1$. Lấy $T_{v_1}, T_{v_2} \in \text{Right}(x)$ và đặt $a = P_{T_{v_1}}, b = P_{T_{v_2}}$. Khi đó $|a|, |b| \geq \text{Maxl}(R(x))$; nếu một trong hai độ dài nhỏ hơn $\text{Maxl}(R(x))$, xâu trong $R(x)$ có độ dài $\text{Maxl}(R(x))$ không thể xuất hiện ở nút tương ứng, mâu thuẫn. Nếu $a \notin R(x)$, theo Bổ đề 3.2 suy ra suffix link của một hậu tố thích hợp của a trở vào lớp của x . Nếu $a \in R(x)$, vì $T_{v_1} \neq T_{v_2}$, ta có $a \neq b$, và $|b| \geq \text{Maxl}(R(x))$, nên một lập luận tương tự cũng cho một xâu có suffix link trở vào $R(x)$. Mệnh đề được chứng minh.

(ii) Biết $x \in S(P_{T_v})$, do đó $|\text{Right}(P_{T_v})| \leq |\text{Right}(x)|$. Vì $|\text{Right}(x)| = 1$, suy ra $\text{Right}(P_{T_v}) = \{T_v\} = \text{Right}(x)$, tức $P_{T_v}Rx$.

(iii) Từ phản đảo của (i), nếu $|\text{Right}(x)| \neq 1$ thì $|\text{Right}(x)| \geq 2$. Theo giả thiết, với mọi $T_v \in \text{Right}(x)$ ta có $|P_{T_v}| > \text{Maxl}(R(x))$. Tồn tại T_{v_1}, T_{v_2} sao cho hậu tố chung dài nhất của $P_{T_{v_1}}$ và $P_{T_{v_2}}$ có độ dài đúng bằng $\text{Maxl}(R(x))$. Nếu không, mọi cặp đều có hậu tố chung dài hơn $\text{Maxl}(R(x))$; theo Bổ đề 2.1(iii), hậu tố chung dài hơn đó cũng tương đương với x , mâu thuẫn với định nghĩa $\text{Maxl}(R(x))$.

Đặt

$$a = (P_{T_{v_1}})_{|P_{T_{v_1}}|-\text{Maxl}(R(x)), |P_{T_{v_1}}|}, \quad b = (P_{T_{v_2}})_{|P_{T_{v_2}}|-\text{Maxl}(R(x)), |P_{T_{v_2}}|}.$$

Từ cách chọn, $a \neq b$, $a \notin S(b)$, $b \notin S(a)$. Theo Bổ đề 2.1(i), $\neg(aRb)$. Dùng Bổ đề 3.2 ta có $s(a)Rx$ và $s(b)Rx$. Lấy $u = a$, $v = b$ là được.

Định lý 4.2. Suffix automaton xây cho một trie có số trạng thái tuyến tính.

Chứng minh. Xét cây parent tạo bởi suffix link. Từ Bổ đề 4.1(i) và (ii), với mọi lá $\Phi(T)_s$ trên cây parent đều tồn tại một tiền tố P_{T_v} sao cho $P_{T_v}Rs$. Từ Bổ đề 4.1(iii), mọi trạng thái $\Phi(T)_s$ mà với mọi T_v đều không có sRP_{T_v} thì phải có ít nhất hai con trong cây parent. Nói cách khác, các lá và các nút chỉ có một con trên cây parent đều tương ứng với một tiền tố nào đó P_{T_v} ; các nút còn lại có ít nhất hai con.

Số tiền tố chỉ là $|T|$, và trạng thái đạt được khi đọc từng P_{T_v} là duy nhất. Vì vậy tổng số lá và nút một con trên cây parent không vượt quá $|T|$; toàn bộ số nút của cây không vượt quá $2|T|$. Do đó số trạng thái là tuyến tính.

Nếu không chỉ số trạng thái mà tổng số hàm chuyển, tức số cạnh, cũng tuyến tính, thì toàn bộ automaton sẽ tuyến tính. Tuy nhiên, khác với suffix automaton của một xâu, tổng số hàm chuyển của suffix automaton trên trie có thể ở cấp số trạng thái nhân với kích thước bảng chữ cái. Ta chứng minh nó không vượt quá cấp này, và đưa ra một trie đạt cùng cấp độ.

Định lý 4.3. Với một trie, chặn trên của số hàm chuyển trong suffix automaton là

$$O((\text{số nút của trie}) \cdot (\text{kích thước bảng chữ cái})).$$

Chứng minh. Gọi trie là T . Theo Định lý 4.2, $|\Phi(T)| \leq 2|T|$. Với mỗi trạng thái $\Phi(T)_s$, số chuyển nhiều nhất là $|A|$. Vì vậy tổng số hàm chuyển không vượt quá $2|T||A|$.

Ta cũng dựng được một trie khiến số hàm chuyển đạt chặn này theo cấp độ. Chỉ cần xét những ký tự thực sự xuất hiện trên cạnh của T , nên $|A| \leq |T|$. Một phần của T là một dây dài $|T|/2$, mọi cạnh trên dây đều mang cùng ký tự a . Các nút còn lại nối từ nút cuối của dây bằng các ký tự đôi một khác nhau; gọi tập ký tự này là B . Khi đó $a^ib \in S(T)$ với $0 \leq i \leq |T|/2$ và $b \in B$, đồng thời hiển nhiên $\neg(a^iRa^j)$ nếu $i \neq j$. Vì thế với mỗi trạng thái $\Phi(T)_{a^i}$, mọi $b \in B$ đều tạo một hàm chuyển. Tổng số hàm chuyển đạt $|T|/2 \cdot |B|$, tức cùng cấp với $|T|/2 \cdot |A|/2$, trùng cấp với chặn trên trong Định lý 4.3.

Hình c. Ví dụ trie trong bản gốc gồm một dây toàn ký tự a , rồi tỏa ra nhiều cạnh ký tự khác nhau, làm tổng số hàm chuyển đạt chặn trên theo cấp độ.

Dù số chuyển khi xây suffix automaton cho trie không luôn tuyến tính, trong phần lớn đề thi và bài toán thực tế, bảng chữ cái không lớn; thường chỉ là chữ cái Latin thường hoặc chữ số. Khi bảng chữ cái là hằng số, số chuyển vẫn tuyến tính, nên đây vẫn là một phương pháp hiệu quả để xử lý các bài toán xâu liên quan đến trie.

3.5 Thuật toán xây dựng

Trong thuật toán xây suffix automaton cho xâu [7, 8], ta dùng một thuật toán tăng dần online: từ suffix automaton của s , xây suffix automaton của sa , với $a \in A$. Khi xây cho trie, ta cũng xét thuật toán tăng dần online. Từ phân tích số trạng thái và số chuyển ở trên, ta biết cận dưới độ phức tạp của thuật toán xây dựng là $|T||A|$.

Giả sử từ trie T , ta thêm vào nút T_v một cạnh nhãn a đi tới nút con mới T_w , thu được trie mới T_N . Đặt $s = P_{T_v}$, nên tiền tố mới là $sa = P_{T_w}$. Xét những phần của $\Phi(T_N)$ cần sửa trên nền $\Phi(T)$. Trước hết phân tích số trạng thái.

3.5.1 Thay đổi trạng thái

Với mọi xâu đã có thể tiếp tục mở rộng thành hậu tố của trie mới, trạng thái tương ứng trong $\Phi(T_N)$ phải tồn tại. Ngược lại, nếu một xâu không thể mở rộng thành hậu tố mới, trạng thái đó không cần xuất hiện thêm. Nếu một lớp tương đương không đổi sau khi thêm T_w , trạng thái suffix automaton tương ứng cũng không đổi, tức $\Phi(T_N)_x = \Phi(T)_x$.

Trường hợp 1. Nếu T_v đã có một con nối bằng cạnh nhãn a , ta không cần sửa $\Phi(T)$.

Trường hợp 2. Nếu $sa \in F(T)$, nghĩa là trong $\Phi(T)$ đã có chuyển nhãn a từ trạng thái của s , thì lớp tương đương $R_T(sa)$ có thể bị tách làm hai lớp.

Thật vậy, với $x, y \in F(T)$ và $xR_T y$, nếu trong tập $Right_{T_N}$ của x và y đều xuất hiện nút mới T_w , thì khi bỏ T_w ra, hai tập vẫn bằng nhau, nên chắc chắn $xR_{T_N} y$. Vì vậy mọi xâu tương đương với sa trong T_N vốn đều nằm trong cùng lớp $R_T(sa)$. Với $x \in R_T(sa)$, nếu $xR_{T_N} sa$, không có trạng thái mới. Nếu tồn tại $x \in R_T(sa)$ nhưng $\neg(xR_{T_N} sa)$, cần tách $R_T(sa)$ thành hai lớp: một lớp tương đương với sa trong T_N , một lớp không tương đương với nó. Lớp không tương đương có tập $Right$ không chứa T_w , nên tập $Right$ của nó giữ nguyên như trong T .

Xét suffix link của hai trạng thái mới. Gọi $u, v \in F(T)$, trong đó $uR_{T_N} sa$, $\neg(vR_{T_N} sa)$, nhưng $uR_T sa$ và $vR_T sa$. Nếu đặt u vào lớp thứ nhất và v vào lớp thứ hai, vì $sa = P_{T_w}$ và $uR_{T_N} sa$, mọi hậu tố của u nhận thêm cùng vị trí mới nên suffix link của lớp thứ nhất không đổi. Với v , phần mới bị tách ra đúng tại lớp của sa , do đó suffix link của lớp thứ hai trở tới lớp mới chứa sa . Nói ngắn gọn, nếu một trạng thái bị tách, một bản sao giữ suffix link cũ, bản sao còn lại có suffix link trở tới trạng thái của sa .

Hình d. Suffix automaton của trie trong Hình a.

Hình e, f, g. Sau khi thêm một nút vào trie của Hình a, bản gốc minh họa lần lượt trie mới, suffix automaton mới và cây parent mới. Nút và cạnh màu cam biểu thị phần mới trong automaton/cây parent.

Khi đã biết lớp tương đương chứa sa trong T_N , mọi hậu tố của sa trước đó đã được đưa vào các lớp liên quan, và các trạng thái ấy không đổi, nên không cần xét lại.

Trường hợp 3. Nếu $sa \notin F(T)$, ta tạo một lớp mới $R_{T_N}(sa)$. Vì xuất hiện một hậu tố mới, vẫn có khả năng một lớp tương đương cũ bị tách.

Có một kết luận khá rõ ràng: $s_{T_N}(sa)$ bằng hậu tố dài nhất của sa có xuất hiện trong $F(T)$. Chứng minh đơn giản: nếu $x \in S(sa)$ nhưng $x \notin F(T)$, thì x chỉ xuất hiện tại nút mới T_w , và sa cũng chỉ xuất hiện tại T_w ; do đó $Right(sa) = Right(x) = \{T_w\}$, nên $xRsa$. Ngược lại, nếu $x \in S(sa)$ và $x \in F(T)$, thì $|Right(x)| > 1$, nên $\neg(xRsa)$.

Kết luận này giúp quá trình xây dựng thuận tiện hơn. Các hậu tố

$$y \in S(sa), \quad y \notin S(s_{T_N}(sa))$$

tạo thành lớp mới $R_{T_N}(sa)$, và ta tạo một trạng thái mới cho lớp đó. Với những $y \in S(s_{T_N}(sa))$, tức phần hậu tố còn lại, có thể xem $s_{T_N}(sa)$ như sa trong Trường hợp 2. Dù Trường hợp 2 dùng tính chất $sa = P_{T_w}$, bản thân $s_{T_N}(sa)$ đã có tính cực đại cần thiết, nên không gây sai lệch.

Bài toán còn lại là tìm $s_{T_N}(sa)$. Theo kết luận trên, ta cần hậu tố dài nhất của sa đã xuất hiện ít nhất một lần trong $F(T)$. Bắt đầu từ trạng thái $\Phi(T)_s$, đi lên cây parent qua các suffix link cho đến khi gặp trạng thái đầu tiên có chuyển nhãn a . Nếu trạng thái đó đại diện bởi một xâu r , thì ra là hậu tố dài nhất của sa xuất hiện ít nhất hai lần, tức $s_{T_N}(sa) = ra$.

Từ ba trường hợp trên, khi thêm một ký tự vào trie, số trạng thái tăng nhiều nhất hai. Điều này một lần nữa chứng minh số trạng thái tuyến tính, đồng thời mô tả được thay đổi của suffix link. Ta vẫn chưa xét cách sửa hàm chuyển; phần tiếp theo xử lý điểm này.

3.5.2 Thay đổi hàm chuyển

Trong Trường hợp 2, nếu không tạo trạng thái mới, chắc chắn không có thay đổi hàm chuyển. Nếu có, lớp cũ $R_T(sa)$ bị tách thành hai lớp $R_{T_N}(sa)$ và một lớp còn lại. Vì nút mới T_w là lá, nó không làm thay đổi các chuyển vốn có từ các xâu cũ. Các chuyển của hai trạng thái mới có thể sao chép từ trạng thái cũ tương ứng.

Tuy nhiên, vì lớp $R_T(sa)$ cũ bị xóa, có thể tồn tại các trạng thái có chuyển nhãn a trở tới trạng thái cũ đó; những chuyển này phải được sửa. Theo phân tích ở Trường hợp 2, các chuyển dọc theo chuỗi suffix link từ trạng thái của s lên trên, chừng nào chúng còn đi tới lớp cũ, sẽ được đổi để trở tới lớp mới phù hợp. Những chuyển khác giữ nguyên. Cách làm trực tiếp là giữ một bản sao đại diện cho phần không đổi, tạo một trạng thái mới cho phần bị tách, rồi bạo lực sửa các chuyển nhãn a trên chuỗi suffix link cho tới khi gặp trạng thái mà chuyển nhãn a không còn trở tới lớp cũ.

Trong Trường hợp 3, ta tạo trạng thái mới $R_{T_N}(sa)$; phần còn lại giống Trường hợp 2. Vì T_w là lá, trạng thái mới ban đầu không có chuyển ra. Do có trạng thái mới, một số trạng thái trên chuỗi suffix link có thể cần thêm chuyển nhãn a trở tới trạng thái mới của sa . Theo phân tích Trường hợp 3, chỉ cần thêm các chuyển này dọc theo chuỗi từ trạng thái của s đi lên cho tới khi gặp trạng thái đã có chuyển nhãn a thích hợp.

3.5.3 Phân tích độ phức tạp

Tương tự trường hợp một xâu, toàn bộ thuật toán là thuật toán tăng dần online. Với trie T , số lần thực hiện là $|T|$. Mỗi lần chỉ thêm nhiều nhất hai trạng thái, nên phân trạng thái là $O(|T|)$. Suffix link của mỗi trạng thái mới chỉ sửa một lần, phần này cũng là $O(|T|)$.

Xét hàm chuyển. Trong Trường hợp 2 có thể cần sao chép một tập chuyển; mỗi trạng thái có nhiều nhất $|A|$ chuyển, nên phần này là $O(|T| |A|)$. Phần phức tạp hơn là các lần bạo lực sửa chuyển khác. Có thể chứng minh tổng thời gian cho phần này không vượt quá tổng độ sâu các lá trong trie; gọi tổng đó là $G(T)$.

Ý tưởng chứng minh giống phân tích thế năng. Gọi $Dep(y)$, với $y \in F(T)$, là độ sâu của trạng thái $\Phi(T)_y$ trong cây parent. Xét đường từ trạng thái $\Phi(T)_s$ và trạng thái mới tới gốc trên cây parent. Theo Bổ đề 2.1(iii), nếu tồn tại lớp của sa thì cũng tồn tại lớp của s , nên $Dep(sa) \leq Dep(s) + 1$. Khi bạo lực sửa chuyển, ta liên tục đặt $c = s(c)$ và sửa chuyển của c ; mỗi bước như vậy làm $Dep(c)$ giảm 1. Khi dừng, độ sâu của trạng thái liên quan chỉ có thể tăng thêm một hằng số so với $Dep(s)$. Vì mỗi lần sửa đều làm giảm thế năng, tổng số lần sửa là cấp độ độ sâu.

Có thể xem Dep là hàm thế năng của trạng thái hiện tại. Với mỗi lá của T , nếu thực hiện riêng quá trình chèn theo đường từ gốc tới lá đó, mỗi phép chèn làm Dep tăng nhiều nhất 3, còn mỗi lần bạo lực sửa chuyển làm Dep giảm 1. Do đó số lần bạo lực sửa chuyển thực chất bị chặn bởi độ sâu; cộng trên mọi lá sẽ được kết luận trên.

Độ phức tạp này chưa phải đẹp, nhưng độ phức tạp của xây dựng online có thể đạt đến cấp đó.

Hình h. Nếu chèn các nút theo thứ tự chỉ số nút trong ví dụ của bản gốc, độ phức tạp đạt $O(G(T))$.

Khi không yêu cầu online, ta có thể thêm các nút trên trie theo thứ tự FIFO, tức thứ tự duyệt rộng. Khi đó chứng minh độ phức tạp của J. A. Blumer và cộng sự [9, 10] cho thuật toán xây trên xâu cũng áp

dụng cho trie. Chỉ những tổ hợp xâu con cố định mới khiến ta phải bạo lực sửa hàm chuyển [6], gợi ý rằng phần độ phức tạp này không vượt quá $O(|T|)$.

Tóm lại, nếu yêu cầu online, độ phức tạp xây dựng là $O(G(T))$. Nếu cho phép offline, ta có thể xây theo thứ tự FIFO và đạt độ phức tạp $O(|T| |A|)$, đúng với cận dưới đã nêu.

3.6 Ứng dụng

3.6.1 ZJOI 2015: Vùng đất huyền tưởng được chú thần ưu ái

Đề bài đã xuất hiện ở phần lời nói đầu, nên không nhắc lại nữa.

Sau chuyển hóa, bài toán trở thành: đếm số xâu con khác nhau về bản chất của một trie. Bằng phương pháp xây suffix automaton cho trie ở trên, ta thu được một automaton có thể nhận ra mọi hậu tố của trie. Mỗi xâu con của trie chắc chắn là tiền tố của một hậu tố nào đó, giống hệt tính chất khi xây suffix automaton cho một xâu. Vì vậy, với một trạng thái $\Phi(T)_s$, số xâu con khác nhau mà nó đại diện là $|R(s)|$; cộng $|R(s)|$ trên mọi trạng thái sẽ cho đáp án.

Cũng có cách khác. Sau khi xây automaton, trực tiếp đếm có bao nhiêu xâu khác nhau không bị automaton từ chối; điều này tương đương đếm số xâu con khác nhau, hay đếm số đường chuyển khác nhau tồn tại khi bắt đầu từ trạng thái đầu. Xem automaton như một đồ thị có hướng không chu trình, ta chỉ cần quy hoạch động đơn giản trên đồ thị này.

Theo phân tích độ phức tạp ở trên, đề bài không yêu cầu online, nên có thể dùng thuật toán offline với độ phức tạp $O(n|A|)$. Thực ra điều kiện số lá là hằng số rất mạnh; có thể chứng minh automaton xây bằng phương pháp trên có thời gian $O(n)$. Lý do là ta có thể tách riêng đường từ gốc đến từng lá thành các xâu, rồi dùng phân tích độ phức tạp giống trường hợp xâu [9]. Vì điều này không liên quan trực tiếp đến nội dung chính của bài viết, tác giả bỏ qua chứng minh chi tiết.

Đến đây bài toán trong phần lời nói đầu đã được giải quyết trọn vẹn.

Từ bài toán này có thể thấy hầu hết bài toán xâu xử lý được bằng suffix automaton đều có thể mở rộng lên trie: tìm xâu con thứ K nhỏ nhất của trie, tìm xâu con chung dài nhất của nhiều trie, đếm số lần xuất hiện của xâu vòng trên trie, v.v. Cũng có những bài rất dễ trên xâu nhưng phức tạp hơn khi mở rộng lên trie, chẳng hạn đếm số xâu khác nhau mà tiền tố của chúng là một xâu cho trước. Các bài này đều có thể giải sau khi xây suffix automaton cho trie.

3.6.2 So sánh với AC automaton

Trong các thuật toán xâu quen thuộc, AC automaton xây một trie từ m mẫu rồi xây các con trỏ fail, sau đó có thể nhận diện số lần xuất hiện của từng mẫu trong xâu đầu vào. Ta có thể thay thế cách dùng này bằng suffix automaton trên trie: xây trie từ m xâu, rồi xây suffix automaton cho trie đó. Khi đọc xâu cần nhận diện, nếu trạng thái hiện tại $\Phi(T)_s$ không có chuyển nhãn a , ta nhảy tới $fa(\Phi(T)_s)$ và tiếp tục nhận diện. Rõ ràng tổng số lần nhảy không vượt quá độ dài xâu đang nhận diện.

Về độ phức tạp, trie T xây từ m xâu có $G(T)$ không vượt quá tổng độ dài của m xâu, nên độ phức tạp không kém hơn AC automaton. Nhưng thuật toán xây ở đây có thể là online, trong khi thuật toán xây AC automaton chỉ là offline.

3.6.3 Cây parent

Hình i. Bản gốc minh họa việc bổ sung các ký tự trên cạnh của Hình b để nhìn cây parent như một trie của các tiền tố đảo.

Thực chất, ta có thể đảo toàn bộ tiền tố trong trie rồi xây lại thành một trie, tương tự suffix tree của một xâu. Với mỗi trạng thái $\Phi(T)_s$, sắp xếp các con của nó trong cây parent. Cụ thể, nếu

$$Fa(\Phi(T)_x) = \Phi(T)_s,$$

ta sắp xếp các trạng thái $\Phi(T)_x$ theo thứ tự của đoạn

$$x_{1, |x| - \text{Maxl}(R(s))}.$$

Khi đó ta thu được một mảng tiền tố của trie, tức thứ tự sau khi đảo mọi tiền tố. Ta cần duy trì cho $\Phi(T)_s$ một nút $T_v \in \text{Right}(s)$; kết hợp nhân đôi trên cây, có thể nhanh chóng tìm được đoạn $x_{1, |x| - \text{Maxl}(R(s))}$.

Với tính chất này, có thể nhanh chóng xác định trạng thái tương ứng khi thêm một ký tự $a \in A$ vào trước xâu đang nhận diện. Giả sử hiện ở trạng thái $\Phi(T)_s$. Ta tìm nhị phân trong các con của nó trên cây parent để tìm $\Phi(T)_{as}$, hoặc kết luận $\Phi(T)_{as} = \Phi(T)_s$.

3.7 Lời cảm ơn

Xin cảm ơn thầy Qu Yunhua của Trường Yali đã lâu dài hướng dẫn và quan tâm tới việc học tập cũng như cuộc sống của tôi.

Xin cảm ơn cha mẹ đã nuôi dưỡng tôi.

Xin cảm ơn các thầy Zhu Quanmin và Wang Xingming của Trường Yali đã chỉ dạy.

Xin cảm ơn CCF đã cung cấp nền tảng và cơ hội giao lưu.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia vì sự tận tụy.

Xin cảm ơn các anh Huang Zhixiang của Đại học Thanh Hoa và Huang Yuyang của Đại học Giao thông Thượng Hải đã giúp đỡ tôi.

Xin cảm ơn các bạn Yang Dingcheng, Kuang Zhengfei và Liu Jiancheng của Trường Yali đã đọc duyệt bài viết này.

Xin cảm ơn tất cả những bạn học từng giúp đỡ tôi.

Tài liệu tham khảo của bài viết

- [1] A. V. Aho and M. J. Corasick, Efficient string matching: An aid to bibliographic research.
- [2] R.-S. Boyer and J. S. Moore, A fast string searching algorithm.
- [3] D. E. Knuth, J. H. Morris and V. I. Pratt, Fast pattern-matching in strings.
- [4] P. Weiner, Linear pattern-matching algorithms.
- [5] E. M. McCreight, A space-economical suffix-tree construction algorithm.
- [6] M. Mohri, P. Moreno and E. Weinstein, General Suffix Automaton Construction Algorithm and Space Bounds.
- [7] M. Crochemore, Transducers and repetitions.
- [8] Chen Lijie, *Suffix automaton*.
- [9] J. A. Blumer, Algorithms for the directed acyclic word graph and related structures.
- [10] A. Blumer, J. Blumer, D. Haussler, R. M. McConnell and A. Ehrenfeucht, Complete inverted files for efficient text retrieval and analysis.
- [11] U. Manber and G. Myers, Suffix arrays: A new method for on-line string searches.

[12] Luo Suijian, *Suffix array: một công cụ mạnh để xử lý xâu*.

[13] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, *Introduction to Algorithms*.

4 Bàn về ứng dụng tư tưởng heuristic trong thi tin học

Ren Zhizhou

Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Thuật toán heuristic là các thuật toán được thiết kế dựa trên kinh nghiệm. So với các thuật toán tối ưu hóa truyền thống, heuristic thường đánh đổi độ chính xác hoặc tính ổn định để lấy tốc độ. Bài viết thử đưa ra một định nghĩa cụ thể hơn cho heuristic trong thi tin học, đồng thời thảo luận một số thuật toán heuristic thường dùng và một số mô hình bài toán. Trong đó có một thuật toán xấp xỉ cho cây Steiner nhỏ nhất với độ phức tạp $O(|S||V|^2)$; bài viết cũng chứng minh tỉ lệ xấp xỉ của thuật toán này đạt

$$2 \left(1 - \frac{1}{\text{leaf}} \right) \leq 2 \left(1 - \frac{1}{|S|} \right),$$

trong đó leaf là số lá của cây Steiner tối ưu.

4.1 Dẫn nhập

Thuật toán heuristic là một lớp thuật toán được nêu ra trong tương quan với các thuật toán tối ưu hóa. Chúng được xây dựng từ trực giác hoặc kinh nghiệm, với mục tiêu tìm được một nghiệm khá tốt trong chi phí thời gian và bộ nhớ chấp nhận được. Khái niệm này rất rộng; đa số heuristic khá phức tạp và không thể đưa thẳng vào bài thi OI.

Tác giả giản lược và cải tiến một số thuật toán heuristic để chúng phù hợp hơn với OI, rồi định nghĩa thêm hai kiểu heuristic trong bối cảnh thi tin học:

- dựa vào trực giác để điều chỉnh và tối ưu thuật toán, hy sinh tính ổn định để đổi lấy tốc độ;
- dựa vào trực giác để trực tiếp dựng nghiệm tốt, hy sinh tính đúng tuyệt đối để giải nhanh.

Nhiều thuật toán quen thuộc cũng chứa tư tưởng heuristic. Sau tối ưu, chúng không làm giảm độ phức tạp xấu nhất, nhưng hiệu suất thực tế thường tăng rõ rệt, chẳng hạn:

- truy vấn điểm gần nhất trên k -d tree;
- Bellman–Ford tối ưu bằng hàng đợi, tức SPFA, và các tối ưu tham lam liên quan;
- mô phỏng tôi luyện, thuật toán di truyền và các thuật toán tìm kiếm ngẫu nhiên lấy cảm hứng từ khoa học tự nhiên.

Bài viết thảo luận ba mô hình heuristic:

- thuật toán A^* và các biến thể, dùng để nối giữa thuật toán xấp xỉ và lời giải tối ưu;
- phân loại Bayes ngây thơ, một thuật toán xác suất kết hợp phán đoán trực quan với suy luận toán học;
- gộp heuristic, một tối ưu đơn giản cho các bài toán cần hợp nhất dữ liệu.

Ngoài việc áp dụng mô hình kinh điển, nhiều khi ta cũng có thể tự thiết kế heuristic. Bài viết dành nhiều dung lượng cho hai bài toán cổ điển:

- bài toán chia đều có trọng số, hay *2-partition problem*, từ đó có một thuật toán xấp xỉ có xác suất đúng rất cao và quy trình thiết kế đáng tham khảo;
- bài toán cây Steiner, được trình bày kỹ hơn, với một thuật toán hiệu quả có thể đánh giá sai số xấp xỉ, rồi kết hợp với A^* để thu được thuật toán xấp xỉ cho K cây Steiner nhỏ nhất.

Ở mỗi phần, tác giả đều đưa ví dụ với hy vọng giúp người đọc nắm được cách sử dụng và tiếp tục phát triển các ý tưởng này.

4.2 Thuật toán A^*

Trong một số bài toán rất khó tìm thuật toán đa thức hiệu quả, ta buộc phải dùng tìm kiếm, trong khi số trạng thái lại rất lớn. Khi đó có thể thiết kế hàm ước lượng để cắt tỉa; phần này trình bày một số cách dùng ước lượng để tăng tốc tìm kiếm.

4.2.1 Tìm kiếm heuristic

Giả sử trạng thái hiện tại là s , chi phí thực tế đã đi tới s là $g(s)$, ước lượng chi phí từ s tới trạng thái đích là $h(s)$, còn chi phí tối ưu thật sự từ s tới đích là $h^*(s)$. Đặt

$$F(s) = g(s) + h(s).$$

Ta có thể dựa vào giá trị này, mỗi lần mở rộng trạng thái có $F(s)$ tốt nhất, và cắt tỉa theo tính chất của hàm ước lượng. Trong bài viết, các bài toán được mặc định là tối thiểu hóa chi phí.

4.2.2 $h(s) = h^*(s)$

Khi hàm ước lượng cho đúng chi phí còn lại, mỗi lần mở rộng trạng thái tốt nhất theo $F(s)$ sẽ nhanh chóng thu được nghiệm tối ưu.

Sau khi đã có nghiệm tối ưu, nếu tiếp tục mở rộng các trạng thái còn lại thì có thể nhận được nghiệm tốt thứ K , đồng thời tránh tính các trạng thái thừa. Phần tổng quát và các tối ưu cho bài toán đặc biệt đã được Yu Yili trình bày trong luận văn đội tuyển quốc gia năm 2014.

4.2.3 $h(s) \leq h^*(s)$

Đôi khi, để tính ước lượng thuận tiện, ta bỏ qua một số ràng buộc trong đề bài. Kết quả thu được là một nghiệm thô “tốt hơn” nghiệm tối ưu thật, tức ước lượng không vượt quá chi phí còn lại. Khi đó ước lượng vẫn cung cấp căn cứ cắt tỉa đúng, giúp giảm phạm vi trạng thái phải xét.

IDA*. IDA*, hay tìm kiếm sâu dần lặp có heuristic, là một biến thể của A^* kết hợp ý tưởng lặp sâu dần với hàm ước lượng. Trong các bài có số trạng thái lớn và khó lưu trữ, cách này tiết kiệm rất nhiều bộ nhớ, đồng thời mã cài đặt cũng gọn.

Với lặp sâu dần đơn giản, mỗi lượt thường chọn một bước tăng cố định và dùng giới hạn độ sâu để cắt tỉa. Với IDA*, ta cắt tỉa bằng ước lượng. Vì $h(s) \leq h^*(s)$, giá trị nhỏ nhất của $g(s) + h(s)$ trong các trạng thái bị cắt có thể dùng để ước lượng đáp án kế tiếp, giảm số lần lặp so với bước tăng cố định.

Ví dụ 1: UVa 10181, 15-Puzzle Problem. Bài toán 15-puzzle mở rộng từ 8-puzzle. Trên bảng 4×4 có 15 ô đánh số và một ô trống; mỗi bước có thể đưa một ô kề ô trống vào vị trí trống. Cần tìm số bước ít nhất để tới trạng thái đích. Dữ liệu bảo đảm mọi trường hợp có lời giải đều tìm được trong không quá 45 bước.

Việc phán đoán vô nghiệm cần phân tích riêng về puzzle số, bài viết không đi sâu. Với phần tìm kiếm, một ước lượng rất đơn giản là tổng khoảng cách Manhattan từ mỗi ô không trống tới vị trí đích. Cách này tương đương với việc bỏ qua ràng buộc “chỉ được đổi chỗ với ô trống”, đồng thời bỏ qua ảnh hưởng của một phép đổi chỗ tới ô còn lại.

Vì đã bỏ bớt ràng buộc, ta có $h(s) \leq h^*(s)$. Với dữ liệu có lời giải trong 45 bước, hiệu quả cắt tỉa và hiệu quả giảm số vòng lặp đều khá tốt.

4.2.4 Anytime algorithm

Lớp này còn được gọi là thuật toán có thể ngắt giữa chừng. Không phải bài nào cũng dễ tìm hàm ước lượng hiệu quả. Đôi khi ta chỉ ước lượng rất thô, không kiểm soát được $h(s) \leq h^*(s)$, nên cũng không thể thu hẹp không gian tìm kiếm một cách đúng đắn. Những bài như vậy thường xuất hiện dưới dạng nộp đáp án. Ta có thể dùng thuật toán cải thiện dần, trong thời gian thi cố gắng thu được nghiệm càng tốt càng tốt.

Ví dụ 2: CTSC 2009 Day 2, trò chơi số $N \times N$. Trò chơi gồm $N^2 - 1$ ô trượt và một ô trống. Mục tiêu là di chuyển ô trống lên, xuống, trái, phải để đưa các ô về một thứ tự quy định. Đây là bài nộp đáp án, với $3 \leq N \leq 6$ và 10 bộ dữ liệu.

So với ví dụ 1, miền dữ liệu lớn hơn nhiều. Đổi lại, bài không yêu cầu nghiệm chính xác; chỉ cần trong thời gian cho phép tìm được nghiệm đủ tốt thì vẫn có điểm đáng kể.

Nếu dùng trực tiếp ước lượng Manhattan của ví dụ 1 rồi chạy IDA*, ta có thể nhanh chóng tìm được nghiệm tối ưu cho 5 bộ nhỏ. Nhưng giá trị ước lượng vẫn cách đáp án thật khá xa, nên các bộ còn lại không xử lý được.

Một cách sửa đơn giản là đặt

$$h'(s) = c h(s),$$

với c là hằng số điều chỉnh theo từng bộ dữ liệu. So với $h(s)$, $h'(s)$ gần số bước thật hơn, nên có thể đưa vào A^* . Tuy vậy, A^* tốn bộ nhớ rất lớn, đó cũng là lý do ví dụ 1 không dùng A^* .

Với bài nộp đáp án, ta có thể thỏa hiệp: chỉ giữ lại một phần trạng thái có ước lượng tốt nhất. Mỗi lần chọn trạng thái tốt nhất để mở rộng; nếu không còn chỗ lưu trạng thái mới thì bỏ trạng thái kém nhất. Vì bài có rất nhiều phương án lời giải, việc loại bỏ một số trạng thái “xấu” ít ảnh hưởng đến đáp án cuối. Ngoại trừ một bộ đặc biệt, cách này có thể đạt khoảng 9 đến 10 điểm mỗi bộ.

4.3 Gộp heuristic

Gộp heuristic là một chiến lược hợp nhất đơn giản, có thể tối ưu hiệu quả cho các thuật toán cần gộp dữ liệu.

4.3.1 Gộp heuristic trên cấu trúc dữ liệu

Giả sử cần gộp hai cấu trúc dữ liệu A, B , số phần tử đang được chúng quản lý lần lượt là a, b và $a \leq b$. Ta tách toàn bộ thông tin trong cấu trúc nhỏ A , rồi lần lượt chèn chúng vào cấu trúc lớn B .

Định lý 3.1. Giả sử sau một số phép gộp ta thu được một cấu trúc dữ liệu có n phần tử. Khi dùng chiến lược gộp heuristic, tổng số lần chèn phát sinh là $O(n \log n)$.

Chứng minh. Xét riêng từng phần tử. Khi một phần tử bị chèn sang cấu trúc khác, kích thước cấu trúc chứa nó sau phép gộp ít nhất gấp đôi trước đó. Sau $O(\log n)$ lần chèn, phần tử đó chắc chắn đã nằm trong cấu trúc cuối cùng. Cộng trên mọi phần tử, ta được $O(n \log n)$.

Chiến lược này có thể gộp các cấu trúc dữ liệu phức tạp, và cũng dễ mở rộng cho các kiểu tập dữ liệu khác.

Ví dụ 3: HNOI 2009, Bánh pudding huyền ảo. Có n chiếc pudding xếp thành một hàng và m thao tác. Mỗi lần đổi toàn bộ pudding của một màu thành màu khác, rồi hỏi hiện có bao nhiêu đoạn màu liên tiếp. Dữ liệu có $n \leq 10^6$.

Ta dùng danh sách liên kết hoặc vector để lưu các vị trí của từng màu. Khi đổi màu, chọn danh sách nhỏ hơn để tách thô, sửa từng vị trí và cập nhật đóng góp vào số đoạn. Theo Định lý 3.1, tổng số lần sửa là $O(n \log n)$, nên có thể bảo trì trực tiếp.

Ví dụ 4: BZOJ 3510, Thủ đô. Cần duy trì một rừng n đỉnh với ba thao tác:

- nối hai đỉnh u, v , bảo đảm sau khi nối vẫn là rừng;
- hỏi trọng tâm của cây chứa đỉnh x , nếu có hai trọng tâm thì lấy chỉ số nhỏ hơn;
- hỏi xor chỉ số trọng tâm của tất cả các cây.

Trước hết xét việc thêm một lá vào một cây. Trọng tâm chỉ có thể di chuyển về phía lá mới, và nhiều nhất di chuyển một lần.

Khi nối hai cây, ta tách thô cây có ít đỉnh hơn và thêm từng đỉnh vào cây lớn dưới dạng lá, đồng thời duy trì trọng tâm. Theo Định lý 3.1, số lần thêm lá là $O(n \log n)$; chỉ cần thêm một cấu trúc dữ liệu để kiểm tra trọng tâm có di chuyển hay không.

4.3.2 Hợp nhất theo hạng trong DSU

Hợp nhất theo hạng là một cách duy trì DSU rất thực dụng: mỗi lần nối gốc của tập nhỏ vào gốc của tập lớn. Từ Định lý 3.1 suy ra chiều cao cây là $O(\log n)$. Khi cài đặt, cũng có thể trực tiếp duy trì độ cao rồi hợp nhất theo hạng.

Cách này bảo đảm độ sâu cây DSU ngay cả khi không nén đường đi. Trong một số bài, nó tránh được việc mất thông tin khi nén đường đi.

Ví dụ 5: NOIP 2013, Vận tải bằng xe tải. Có n thành phố, m đường hai chiều; mỗi đường có giới hạn trọng lượng z_i . Với q xe tải, cần biết mỗi xe có thể chở tối đa bao nhiêu mà không vượt quá giới hạn trên đường đi. Dữ liệu:

$$n \leq 10000, \quad m \leq 50000, \quad q \leq 30000, \quad z_i \leq 100000.$$

Cách thông thường là dùng Kruskal dựng cây khung lớn nhất. Giá trị nhỏ nhất trên đường đi trong cây khung lớn nhất chính là giới hạn lớn nhất có thể giữa hai đỉnh; có thể trả lời bằng nhân đôi, với độ phức tạp $O(m \log m + q \log n)$.

Nếu dùng hợp nhất theo hạng thay cho nén đường đi, ta có thể ghi lại với mỗi đỉnh trong DSU trọng số cạnh nối nó với cha. Giá trị nhỏ nhất trên đường trong cây DSU vẫn biểu diễn giới hạn rộng nhất giữa

hai đỉnh. Trả lời thô có độ phức tạp $O(m \log m + q \log n)$, nhưng cài đặt ngắn hơn cách dựng cây rồi nhân đôi.

Vì bài toán chuyển hóa vẫn là truy vấn giá trị nhỏ nhất trên đường đi, ta vẫn có thể dùng nhân đôi; về lý thuyết phần hồi đạt $O(\log \log n)$. Nếu chọn thuật toán sắp xếp tuyến tính và vẫn nén đường đi trong DSU theo cách nối cạnh ban đầu, độ phức tạp lý thuyết có thể đạt

$$O(n + m\alpha(n) + q \log \log n).$$

4.4 Bài toán trừu tượng và học máy

Khi gặp các bài toán phân loại trừu tượng, cảm giác trực quan thường không cho căn cứ phán đoán chính xác. Khi đó có thể dùng học máy thay cho quan sát thủ công. Phần này thông qua một bài Google Code Jam để giới thiệu ngắn gọn một thuật toán phân loại thực dụng.

4.4.1 Bayes ngây thơ

Công thức Bayes. Ký hiệu $P(A | B)$ là xác suất sự kiện A xảy ra khi đã biết B xảy ra; ký hiệu $P(A \cap B)$ là xác suất cả A và B cùng xảy ra.

Định lý 4.1.

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}.$$

Chứng minh. Ta có

$$P(A \cap B) = P(A)P(B | A) = P(B)P(A | B).$$

Chuyển về sẽ thu được công thức Bayes.

Phán đoán ngây thơ. Gọi S là sự kiện cần phân loại, A và B là hai lớp có thể chọn. Nếu $P(A | S) > P(B | S)$, ta cho rằng S thuộc lớp A ; ngược lại thuộc lớp B . Khi có nhiều lớp hơn, chọn lớp có xác suất lớn nhất.

Giả định ngây thơ. Giả sử sự kiện cần phân loại có các thuộc tính

$$S = \{S_1, S_2, \dots, S_m\}.$$

Nếu giả định các thuộc tính đặc trưng này độc lập với nhau khi đã biết lớp, ta có thể xấp xỉ

$$P(S | A) \approx \prod_{i=1}^m P(S_i | A).$$

So sánh các xác suất xấp xỉ như vậy sẽ cho kết quả phân loại. Các giá trị $P(S_i | A)$ có thể tính bằng lấy mẫu ngẫu nhiên hoặc bằng một thuật toán riêng cho bài toán.

4.4.2 Ví dụ 6: GCJ Round 1A 2014, *Proper Shuffle*

Có hai cách sinh hoán vị. Đề cho 120 hoán vị độ dài 1000, mỗi hoán vị được sinh độc lập bởi một trong hai thuật toán. Cần nhận diện hoán vị nào do thuật toán nào sinh ra; nếu trả lời đúng ít nhất 109 trong 120 hoán vị thì được chấp nhận.


```

Algorithm 1 GOOD
for k = 0 to n - 1 do
    a[k] = k
end for
for k = 0 to n - 1 do
    p = random_int(k..n - 1)
    swap(a[k], a[p])
end for

```

```

Algorithm 2 BAD
for k = 0 to n - 1 do
    a[k] = k
end for
for k = 0 to n - 1 do
    p = random_int(0..n - 1)
    swap(a[k], a[p])
end for

```

Thuật toán GOOD sinh mọi hoán vị với xác suất bằng nhau, còn BAD thì không.

Phân tích đơn giản. Với GOOD, sau khi quyết định vị trí k , xác suất xuất hiện của mọi giá trị tại vị trí đó đều là $1/n$, và các bước sau không ảnh hưởng vị trí đó.

Với BAD, khi quyết định phép đổi tại vị trí k , nếu $\text{swap}(a[k], a[p])$ có $p < k$, giá trị vốn ở vị trí $a[k]$ bị đổi tới một vị trí cao hơn, rồi sau đó còn có thể tiếp tục bị đổi tới vị trí cao hơn nữa. Điều này tạo ảnh hưởng tinh vi tới xác suất một giá trị xuất hiện tại từng vị trí.

Một đặc trưng đơn giản. Gọi S là hoán vị đầu vào. Có thể dùng số lượng vị trí thỏa $S_i < i$ làm đặc trưng. Phân tích của cuộc thi cho thấy phân bố đặc trưng này giữa GOOD và BAD khác nhau rõ rệt; nếu dùng một ngưỡng thích hợp thì tỉ lệ đúng có thể vượt 90%.

Phân loại Bayes ngây thơ. Ta muốn tính $P(\text{GOOD} \mid S)$. Theo Định lý 4.1:

$$P(\text{GOOD} \mid S) = \frac{P(S \mid \text{GOOD})P(\text{GOOD})}{P(S \mid \text{GOOD})P(\text{GOOD}) + P(S \mid \text{BAD})P(\text{BAD})}.$$

Vì $P(\text{GOOD}) = P(\text{BAD})$, công thức rút gọn thành

$$P(\text{GOOD} \mid S) = \frac{P(S \mid \text{GOOD})}{P(S \mid \text{GOOD}) + P(S \mid \text{BAD})},$$

và tương tự

$$P(\text{BAD} \mid S) = \frac{P(S \mid \text{BAD})}{P(S \mid \text{GOOD}) + P(S \mid \text{BAD})}.$$

Nếu $P(\text{GOOD} \mid S) > P(\text{BAD} \mid S)$, xác suất S do GOOD sinh ra lớn hơn. Điều kiện này tương đương

$$P(S \mid \text{GOOD}) > P(S \mid \text{BAD}).$$

Với GOOD, mọi hoán vị cùng xác suất:

$$P(S \mid \text{GOOD}) = \frac{1}{n!}.$$

Việc tính chính xác $P(S \mid \text{BAD})$ lại rất khó, nên ta xấp xỉ:

$$P(S \mid \text{BAD}) \approx \prod_{k=0}^{n-1} P(S_k \mid \text{BAD}).$$

Để so sánh công bằng hơn, ta cũng dùng xấp xỉ tương tự cho GOOD:

$$P(S \mid \text{GOOD}) \approx \prod_{k=0}^{n-1} \frac{1}{n}.$$

Nếu $P_{i,j}$ là xác suất sau khi chạy BAD ta có $S_i = j$, ta dễ thiết kế một DP $O(n^2)$ để tính toàn bộ bảng này. Từ đó có thể tính $P(S \mid \text{BAD})$ xấp xỉ rồi so sánh với $P(S \mid \text{GOOD})$. Khi tính trực tiếp xác suất có thể gặp yêu cầu độ chính xác cao; thực tế có thể so sánh $P(S \mid \text{BAD}) n^n$ với 1.0, hoặc dùng ngưỡng khác vì bản thân quá trình đã là xấp xỉ.

4.5 Bài toán chia đều có trọng số

Ví dụ 7. Cho một đa tập số nguyên dương S . Ký hiệu $f(S)$ là tổng các phần tử trong S . Chia S thành hai tập S_1, S_2 , với $S_1 + S_2 = S$, sao cho $|f(S_1) - f(S_2)|$ nhỏ nhất, đồng thời đưa ra phương án chia.

4.5.1 Ví dụ đơn giản

Với $S = \{3, 1, 1, 2, 2, 1\}$, có hai cách chia hợp lệ làm $f(S_1) = f(S_2) = 5$, tức $|f(S_1) - f(S_2)| = 0$:

$$S_1 = \{1, 1, 1, 2\}, \quad S_2 = \{2, 3\};$$

$$S_1 = \{3, 1, 1\}, \quad S_2 = \{2, 2, 1\}.$$

4.5.2 Một thuật toán quy hoạch động

Có thể thiết kế DP giống ba lô: đặt $d[i][j]$ cho biết sau khi dùng i phần tử đầu tiên, có thể đạt $f(S_1) = j$ hay không. Độ phức tạp là $O(nf(S))$.

Khi mọi phần tử trong S đều nhỏ, đây đúng là thuật toán tốt. Nhưng thực chất đây chỉ là thuật toán giả đa thức.

4.5.3 Một ý tưởng tham lam đơn giản

Ý tưởng tự nhiên là đọc lần lượt từng phần tử và đặt nó vào tập hiện có tổng nhỏ hơn. Để hai tổng dần áp sát nhau, ta sắp xếp các phần tử giảm dần trước khi xử lý; độ phức tạp là $O(n \log n)$.

Thuật toán này rất tự nhiên và kết quả thường không tệ, nhưng phân tích kỹ sẽ thấy nó có lỗ hổng rõ ràng.

4.5.4 Thuật toán sai phân

Thuật toán sai phân cải tiến thuật toán tham lam ở trên. Ý tưởng ngắn gọn, nhưng chất lượng nghiệm xấp xỉ tốt hơn nhiều.

Tư tưởng cơ bản. Khi đặt hai số X, Y vào hai tập khác nhau, phần ảnh hưởng tới đáp án chỉ là độ lớn của hiệu giữa chúng. Điều này tương đương tạo ra một số mới $|X - Y|$, rồi tiếp tục xem nó như một số thường trong thuật toán.

Nhìn lại thuật toán tham lam trước đó, xét trạng thái ở mỗi quyết định. Giả sử phần tử mới là X , còn các phần tử đã quyết định tạo ra sai phân Y . Việc đưa X vào tập có tổng nhỏ hơn tương đương thay X, Y bằng $|X - Y|$. Nhưng mục đích ban đầu của ta là xử lý phần tử lớn trước, nên quyết định theo thứ tự đọc vào là chưa tối ưu.

Quy trình thuật toán. Theo tư tưởng sai phân, ta dùng heap và DSU để bảo trì:

1. Đưa mọi phần tử của S vào heap. Trong DSU, mỗi phần tử ban đầu là một tập riêng.
2. Lấy hai phần tử lớn nhất khỏi heap, gọi là X, Y ; trong DSU chúng tương ứng với hai tập S_X, S_Y . Quyết định đặt S_X và S_Y ở hai phía khác nhau, đưa $|X - Y|$ trở lại heap, và nối hai tập trong DSU bằng một cạnh trọng số 1 biểu diễn việc đổi phía.
3. Nếu heap chỉ còn một phần tử thì dừng, ngược lại lặp bước trước.
4. Chọn gốc của cây DSU thuộc S_1 . Các phần tử khác dựa vào tính chẵn lẻ của độ dài đường tới gốc để quyết định thuộc S_1 hay S_2 : chẵn cùng phía, lẻ khác phía.

Khi cài đặt DSU, có thể duy trì tính chẵn lẻ của độ dài đường trong lúc nén đường đi, hoặc dùng hợp nhất theo hạng để bảo đảm chiều cao $O(\log n)$.

Hiệu quả xấp xỉ. Thuật toán có độ phức tạp $O(n \log n)$, rất hiệu quả; điều quan trọng là nghiệm có đủ tốt hay không.

Khi n không quá nhỏ, chẳng hạn $n > 1000$, với tập số sinh ngẫu nhiên $S_i \in [0, 2^{30})$, thuật toán gần như chắc chắn chia được S_1, S_2 sao cho

$$|f(S_1) - f(S_2)| \leq 1.$$

Nói cách khác, nếu $f(S)$ lẻ thì sai khác bằng 1, còn nếu $f(S)$ chẵn thì sai khác bằng 0. Có thể thấy xác suất đúng của thuật toán này rất cao.

Khi n nhỏ, có thể dùng thuật toán này để thiết kế hàm ước lượng rồi chạy tìm kiếm heuristic nhằm thu được nghiệm tốt hơn.

4.5.5 Ví dụ 8: POI 2013, *Polarization*

Cho một cây n đỉnh. Hãy định hướng mỗi cạnh thành cạnh có hướng. Nếu từ đỉnh u đi được tới đỉnh v , gọi (u, v) là một cặp hợp lệ. Cần tìm số cặp hợp lệ nhỏ nhất và lớn nhất trong mọi cách định hướng. Dữ liệu có $n \leq 250000$.

Chuyển hóa bài toán. Cây là đồ thị hai phía, nên giá trị nhỏ nhất chắc chắn là $n - 1$.

Với giá trị lớn nhất, nghiệm tối ưu luôn tồn tại một đỉnh x sao cho mọi đỉnh còn lại hoặc có đường đi tới x , hoặc có đường đi từ x tới nó. Phân tích tiếp cho thấy x phải chọn ở trọng tâm của cây.

Lấy x làm gốc, cây tách thành một số cây con. Bài toán còn lại là chia các cây con này thành hai tập sao cho tích kích thước của hai tập lớn nhất.

Thuật toán tối ưu truyền thống. Chú ý rằng tổng các kích thước ở đây là n , nên số giá trị phần tử khác nhau chỉ là $O(\sqrt{n})$. Ta có thể chuyển thành bài toán ba lô đa trọng, một bài toán kinh điển không trình bày lại ở đây. Độ phức tạp là $O(n\sqrt{n})$.

Thuật toán xấp xỉ. Nếu dùng trực tiếp thuật toán sai phân, có thể qua khoảng 80% dữ liệu. Nguyên nhân chính là khi số phần tử ít, sai lệch của nghiệm xấp xỉ lớn hơn.

Ta đặt một ngưỡng. Khi số cây con nhỏ hơn ngưỡng này, dùng ba lô vét cạn; các trường hợp còn lại dùng thuật toán sai phân. Cách ghép này có thể qua toàn bộ dữ liệu chính thức của POI, với độ phức tạp $O(n \log n)$.

Khi thuật toán tối ưu truyền thống đủ nhanh, không nên ưu tiên thuật toán xấp xỉ. Ví dụ này chỉ nhằm cho thấy thuật toán sai phân có thể tạo ra nghiệm xấp xỉ rất tốt.

4.6 Bài toán cây Steiner nhỏ nhất

Ví dụ 9. Cho đồ thị vô hướng có trọng số $G = (V, E)$. Tập đỉnh đặc biệt S là một tập con của V . Cần tìm một đồ thị liên thông

$$T = (V_T, E_T)$$

sao cho $S \subseteq V_T \subseteq V$, $E_T \subseteq E$, $|E_T| = |V_T| - 1$, và tổng trọng số các cạnh trong E_T là nhỏ nhất.

4.6.1 Một thuật toán tối ưu

Cây Steiner nhỏ nhất có thuật toán trạng thái nén kinh điển, chuyển về mô hình đường đi ngắn nhất; có thể xem luận văn đội tuyển quốc gia năm 2014 của Jiang Jinye. Độ phức tạp là

$$O(|V| 3^{|S|} + |E| 2^{|S|}),$$

đây là thuật toán tốt nhưng chỉ áp dụng được cho miền dữ liệu nhỏ.

4.6.2 Một ý tưởng tham lam đơn giản

Một tham lam đơn giản là tìm cây khung nhỏ nhất của đồ thị gốc, rồi xóa các cạnh thừa trên cây khung, tức xóa nhiều nhất có thể nhưng vẫn giữ cho tập S liên thông.

Thuật toán này có lỗi hổng lớn. Trong ví dụ của bản gốc, với

$$V = \{A, B, C, 1, 2, 3, 4, 5\}, \quad S = \{A, B, C\}, \quad x > 2,$$

tham lam theo cây khung nhỏ nhất chọn các cạnh trọng số x và cho nghiệm kém, trong khi nghiệm tối ưu đi qua các cạnh trọng số 1 và các cạnh $x + 1$ như hình minh họa.

4.6.3 Thuật toán heuristic

Dựa trên ví dụ trên, ta cải tiến có chủ đích.

Phân tích thêm. Trong ví dụ, nếu tạm bỏ qua các điểm 1 và 2, rồi xét khoảng cách ngắn nhất giữa các điểm còn lại, cây khung nhỏ nhất trên đồ thị rút gọn đã có thể cho nghiệm tối ưu. Để tổng quát hơn, ta loại cả những điểm không thuộc S khỏi đồ thị rút gọn.

Vấn đề là các đường đi ngắn nhất giữa các cặp đỉnh của S có thể giao nhau. Khi cuối cùng tạo cây, các cạnh giao nhau không được tính lặp.

Quy trình thuật toán. Sau khi chỉnh lý, ta có thuật toán sau:

1. Từ đồ thị gốc $G = (V, E)$, dựng đồ thị đầy đủ $G_1 = (S, E_1)$. Trọng số cạnh giữa hai đỉnh trong S là độ dài đường đi ngắn nhất giữa chúng trong G .
2. Tìm cây khung nhỏ nhất T_1 của G_1 . Nếu có nhiều nghiệm thì chọn tùy ý.
3. Dựng một đồ thị con $G_2 = (V_2, E_2)$ của G : với mỗi cạnh của T_1 , đưa vào G_2 một đường đi ngắn nhất tương ứng trong G . Nếu có nhiều đường đi ngắn nhất thì chọn tùy ý.
4. Tìm cây khung nhỏ nhất T_2 của G_2 . Nếu có nhiều nghiệm thì chọn tùy ý.
5. Xóa khỏi T_2 các cạnh vô dụng sao cho mọi lá còn lại đều là đỉnh thuộc S . Kết quả là cây Steiner T .

Phân tích từng bước:

- Dựng G_1 cần chạy đường đi ngắn nhất một nguồn $|S|$ lần, độ phức tạp $O(|S||V|^2)$ nếu dùng cách cơ bản.
- Tìm cây khung nhỏ nhất của G_1 ; vì G_1 là đồ thị đầy đủ, nên dùng Prim với độ phức tạp $O(|S|^2)$.
- Khi dựng G_2 , mỗi đường đi ngắn nhất có thể qua $O(|V|)$ đỉnh, nhưng tổng số cạnh đường đi được thêm vẫn bị chặn bởi $O(|V|)$ theo phân tích của bản gốc cho trường hợp này.
- Tìm cây khung nhỏ nhất của G_2 , độ phức tạp $O(|V|^2)$ hoặc $O(|V| \log |V|)$ tùy cài đặt.
- Xóa cạnh vô dụng trong $O(|V|)$.

Có thể thấy thuật toán chạy rất hiệu quả, đồng thời chất lượng xấp xỉ cũng tốt. Phần tiếp theo chúng minh chặn trên tương đối cho nghiệm xấp xỉ của nó. Có thể thay bước đường đi ngắn nhất bằng thuật toán hiệu quả hơn để tối ưu thêm độ phức tạp.

Hiệu quả xấp xỉ. Gọi D_T là tổng trọng số cây Steiner T do thuật toán trên thu được, và $D_{\min}$ là tổng trọng số cây Steiner nhỏ nhất $T_{\min}$. Ta sẽ chặn tỉ số $D_T/D_{\min}$.

Bổ đề 6.1. Với một cây T có $m > 1$ cạnh, tồn tại một vòng

$$u_0, u_1, \dots, u_{2m} = u_0$$

thỏa các tính chất:

- mỗi cạnh của T xuất hiện đúng hai lần trong vòng;
- mỗi lá của T chỉ xuất hiện một lần trong vòng, không tính việc u_0 lặp lại ở cuối;
- nếu u_i và u_j , với $i < j$, là hai lá liên tiếp trên vòng, và giữa chúng không có lá nào khác, thì $u_i, u_{i+1}, \dots, u_j$ là một đường đi đơn trên T .

Chứng minh. Chọn một đỉnh làm gốc, rồi duyệt Euler trên cây T . Dãy Euler thu được thỏa các tính chất trên nếu xem mỗi cạnh vô hướng như hai cạnh có hướng ngược nhau.

Định lý 6.1. Gọi leaf là số lá của cây Steiner nhỏ nhất $T_{\min}$. Khi đó

$$\frac{D_T}{D_{\min}} < 2 \left(1 - \frac{1}{\text{leaf}} \right) \leq 2 \left(1 - \frac{1}{|S|} \right).$$

Chứng minh. Vì $\text{leaf} \leq |S|$, bất đẳng thức bên phải là hiển nhiên.

Theo Bổ đề 6.1, trong $T_{\min}$ có thể tạo một vòng L bằng cách đi mỗi cạnh hai lần. Nếu trọng số vòng là D_L , thì $D_L = 2D_{\min}$.

Trên vòng L , chọn đường đơn dài nhất giữa hai lá liên tiếp rồi xóa nó đi; phần còn lại là một đường đi P . Vì có leaf đoạn giữa các lá, đoạn dài nhất có trọng số ít nhất D_L/leaf , do đó

$$D_P \leq \left(1 - \frac{1}{\text{leaf}}\right) D_L = 2 \left(1 - \frac{1}{\text{leaf}}\right) D_{\min}.$$

Trong G_1 , cạnh giữa hai đỉnh thuộc S luôn lấy đường đi ngắn nhất trong G , nên cây khung nhỏ nhất trên G_1 không nặng hơn đường P . Các bước khử trùng cạnh và xóa cạnh vô dụng sau đó không làm tăng tổng trọng số. Vì vậy

$$D_T \leq D_P < 2 \left(1 - \frac{1}{\text{leaf}}\right) D_{\min}.$$

Định lý 6.2. Khi $\text{leaf} > 2$, tỉ lệ xấu nhất đạt đúng

$$\frac{D_T}{D_{\min}} = 2 \left(1 - \frac{1}{\text{leaf}}\right).$$

Chứng minh. Dựng đồ thị $G = (V, E)$ như sau:

$$V = \{v_1, v_2, \dots, v_{\text{leaf}}, v_{\text{leaf}+1}\}, \quad S = \{v_1, v_2, \dots, v_{\text{leaf}}\}.$$

Đặt trọng số

$$d(v_i, v_j) = \begin{cases} 2, & j = i + 1, i < \text{leaf}, \\ 1, & i \leq \text{leaf}, j = \text{leaf} + 1, \\ \infty, & \text{các trường hợp khác.} \end{cases}$$

Trong trường hợp xấu nhất, thuật toán có thể chọn đường đi nối các đỉnh $v_1, v_2, \dots, v_{\text{leaf}}$ thành một chuỗi với tổng trọng số $2(\text{leaf} - 1)$. Trong khi đó, cây Steiner tối ưu là ngôi sao qua $v_{\text{leaf}+1}$, có tổng trọng số leaf . Do đó

$$\frac{D_T}{D_{\min}} = \frac{2(\text{leaf} - 1)}{\text{leaf}} = 2 \left(1 - \frac{1}{\text{leaf}}\right),$$

đúng bằng chặn trên.

4.6.4 Ví dụ 10: kiểm tra lẫn nhau đội tuyển 2015, *Marketing network*

Cho đồ thị $G = (V, E)$. Cần tìm K rừng sinh nhỏ nhất đầu tiên, với ràng buộc tập đỉnh đặc biệt S phải liên thông trong rừng sinh. Dữ liệu:

$$|V|, K < 50, \quad |E| < 100, \quad |S| < 15,$$

cạnh, trọng số cạnh và tập S đều được sinh ngẫu nhiên.

Thuật toán tối ưu. Bài toán nghiệm tốt thứ k có thể dùng A^* . Ta mô tả trạng thái bằng việc mỗi cạnh đã được chọn hay chưa; khi tìm kiếm, quyết định cạnh tiếp theo có được chọn hay không.

Nếu dùng thuật toán trạng thái nén cho cây Steiner làm hàm ước lượng, ta có $h(s) = h^*(s)$. Mỗi lần mở rộng trạng thái tốt nhất. Vì yêu cầu là rừng sinh, sau mỗi $O(n)$ quyết định hiệu lực có thể tìm được nghiệm tốt nhất hiện tại; tìm K nghiệm đầu chỉ cần $O(Km)$ lần tính ước lượng.

Thuật toán này bảo đảm tính đúng và độ phức tạp, nhưng hiệu suất thực tế thấp, không đủ cho miền dữ liệu của bài này.

Thuật toán xấp xỉ. Tiếp tục dùng tư tưởng A^* , nhưng lấy thuật toán heuristic ở phần trước làm ước lượng. Bất lợi là giá trị ước lượng có thể lớn hơn $h^*(s)$, nên không còn thể cất tĩa một cách chắc chắn.

Ta xử lý xấp xỉ bằng cách chọn một hằng số c , dùng $ch(s)$ làm tham khảo cho cận dưới, rồi cất các trạng thái có ước lượng kém. Điều chỉnh c cho phép cân bằng giữa tốc độ chạy và độ chính xác.

4.7 Lời cảm ơn

Xin cảm ơn Hiệp hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và giao lưu.

Xin cảm ơn các thầy Chen Heli, Dong Yehua, Shao Hongxiang và You Guanghui của Trường Trung học số 1 Thiệu Hưng đã nhiều năm quan tâm và hướng dẫn tôi.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Yu Linyun và Chen Xuji đã chỉ dẫn.

Xin cảm ơn các anh Yu Yili, Dong Anhua và He Qizheng của Đại học Thanh Hoa đã giúp đỡ tôi.

Xin cảm ơn các bạn Zhang Hengjie, Wang Jianhao và Jia Yuekai của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở cho tôi.

Xin cảm ơn các thầy cô và bạn học khác từng giúp đỡ, gợi ý cho tôi.

Tài liệu tham khảo của bài viết

- [1] Wikipedia, Heuristic (computer science).
- [2] Wikipedia, Partition problem.
- [3] Wikipedia, Steiner tree problem.
- [4] L. Kou, G. Markowsky and L. Berman, “A fast algorithm for Steiner trees”, *Acta Informatica*.
- [5] Jiang Jinye, *Công cụ mạnh cho giải lập: tối ưu và ứng dụng thuật toán SPFA*, luận văn đội tuyển quốc gia 2009.
- [6] Zhou Erjin, *Bàn về ứng dụng hàm ước lượng trong thi tin học*, luận văn đội tuyển quốc gia 2009.
- [7] Yu Yili, *Một số phương pháp tìm nghiệm tốt thứ k* , luận văn đội tuyển quốc gia 2014.

5 Bàn về một số phương pháp khớp chuỗi

Wang Jianhao

Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Bài toán khớp chuỗi là một bài toán kinh điển trong thi tin học. Bài viết phân loại và tổng kết các cách giải cho lớp bài toán này. Trước hết tác giả điểm lại một số cấu trúc và thuật toán thường dùng, sau đó tập trung giới thiệu một cấu trúc khớp chuỗi mới là *Border Tree*, từ đó rút ra một cách giải tổng quát cho một lớp bài toán khớp chuỗi. Mỗi thuật toán và kỹ thuật đều đi kèm ví dụ để thuận tiện cho việc hiểu và phân tích.

5.1 Dẫn nhập

Bài toán khớp chuỗi thường xuất hiện trong xử lý văn bản. Trong thời đại dữ liệu lớn, nhiều lĩnh vực phải xử lý các thao tác đa dạng trên lượng dữ liệu lớn. Bài toán khớp chuỗi trong thi tin học là một mô hình

tiêu biểu của kiểu vấn đề đó; bài viết này giới thiệu một số cách giải cho chúng.

Mục 2 nhắc lại các định nghĩa về chuỗi để thuận tiện cho phần sau. Tiếp đó, bài viết tổng kết ngắn gọn một số cấu trúc và thuật toán, chia chúng thành hai nhóm: cách giải dựa trên tiền tố và cách giải dựa trên hậu tố.

Mục 3 giới thiệu các cấu trúc, thuật toán tiền tố gồm trie, KMP và AC automaton. Mục 4 giới thiệu các cấu trúc hậu tố gồm suffix array, suffix tree, suffix cactus và suffix automaton. Mục 5 trình bày chi tiết cấu trúc mới Border Tree. Mục 6 dùng Border Tree để đưa ra một cách giải tổng quát cho một lớp bài toán khớp chuỗi.

Nội dung ở Mục 3 và Mục 4 khá phổ biến, một số thuật toán cũng đã từng được trình bày trong các luận văn đội tuyển quốc gia trước đây, nên ở đây tác giả chỉ tóm tắt và phân tích một số bài toán kinh điển. Mục 5 và Mục 6 được trình bày kỹ hơn. Border Tree là một cấu trúc tương đối mới; các ứng dụng nêu ra trong bài viết nhằm gợi mở để độc giả quan tâm tiếp tục nghiên cứu.

5.2 Định nghĩa

Để tiện mô tả, trước hết nêu một số định nghĩa.

Định nghĩa 2.1 (tiền tố, hậu tố, xâu con, Border, LBorder). Xét xâu $s = s_1s_2 \dots s_n$ có độ dài n .

Với $1 \leq i \leq j \leq n$, ký hiệu $s[i : j] = s_is_{i+1} \dots s_j$. Khi $i > j$, quy ước $s[i : j]$ là xâu rỗng. Nếu $u = s[i : j]$, ta gọi u là một xâu con của s .

Với $1 \leq i \leq n$, $s[i : n]$ là một hậu tố của s , ký hiệu $\text{suf}[i]$; $s[1 : i]$ là một tiền tố của s , ký hiệu $\text{pre}[i]$.

Với $1 \leq i < n$, nếu $s[1 : i] = s[n - i + 1 : n]$, thì $s[1 : i]$ là một *Border* của s . Thông thường xâu rỗng cũng được xem là một Border. Trong mọi Border của s , Border dài nhất được gọi là *LBorder* của s . Tương tự có thể nói $\text{pre}[i]$ có LBorder, hay một xâu con có LBorder.

Định nghĩa 2.2 (xâu tuần hoàn, gốc, độ dài chu kỳ). Với xâu $s = s_1s_2 \dots s_n$, nếu LBorder của s có độ dài L , thì $s[L + 1 : n]$ được gọi là một gốc của s . Nếu $2L > n$, ta gọi s là xâu tuần hoàn. Khi s là xâu tuần hoàn, độ dài của gốc được gọi là độ dài chu kỳ.

Định nghĩa 2.3 (xâu lặp). Nếu một xâu t nhận được bằng cách lặp một xâu s nguyên số lần $k > 1$, thì t được gọi là xâu lặp, ký hiệu $t = s^k$. Rõ ràng xâu lặp là một lớp đặc biệt của xâu tuần hoàn.

5.3 Các cách giải theo tiền tố

Trong các phương pháp khớp chuỗi của thi tin học, có một số cấu trúc và thuật toán dựa trên việc tận dụng hiệu quả thông tin tiền tố để khớp nhanh. Bài viết gọi chúng là nhóm cách giải theo tiền tố. Mục này giới thiệu ba cấu trúc tiền tố và các biến thể của chúng.

5.3.1 Trie

Trie là cây tra cứu từ; dưới đây gọi là cây chữ cái. Cây chữ cái hỗ trợ tìm kiếm và khớp nhiều xâu mẫu. Định nghĩa và phương pháp xây dựng có thể xem trong luận văn đội tuyển quốc gia năm 2004 của Zhu Zeyuan hoặc luận văn năm 2009 của Dong Huaxing.

Cây chữ cái là một cây có gốc, mỗi cạnh biểu diễn một ký tự. Ghép các ký tự trên đường đi từ gốc tới một đỉnh theo thứ tự duyệt sẽ thu được một xâu w . Vì vậy mỗi đỉnh trên cây chữ cái biểu diễn một xâu; gốc biểu diễn xâu rỗng.

Thông thường, khi khớp nhiều xâu mẫu, ta chèn từng xâu mẫu vào cây chữ cái. Theo định nghĩa của trie, mọi tiền tố của mỗi xâu mẫu đều được một đỉnh biểu diễn. Nhờ đó ta có thể dùng tính chất cây để giải nhanh nhiều bài toán khớp chuỗi, chẳng hạn tính nhanh tiền tố chung dài nhất của hai xâu mẫu.

Ví dụ 1 (USACO 2012 Dec Gold, First). Cho m xâu. Thứ tự từ điển của 26 chữ cái thường có thể được chọn tùy ý. Hỏi mỗi xâu có thể trở thành xâu nhỏ nhất theo thứ tự từ điển trong một cách chọn thứ tự chữ cái nào đó hay không. Các xâu đôi một khác nhau.

Dữ liệu:

$$1 < m < 30000, \quad 1 < \text{tổng độ dài các xâu} < 300000.$$

Giới hạn thời gian là 1 giây.

Trước hết xây trie cho m xâu. Với xâu thứ i , giả sử đỉnh u của trie biểu diễn xâu đó. Để xâu thứ i có thể là xâu nhỏ nhất theo một thứ tự từ điển nào đó, trước hết không được có xâu khác là tiền tố của nó. Ngoài ra, trên đường từ gốc tới u , mỗi ký tự được chọn phải nhỏ hơn mọi ký tự khác ở cùng tầng có thể gây cạnh tranh.

Với mỗi xâu, lấy các ràng buộc này ra rồi kiểm tra chúng có mâu thuẫn hay không. Khi cài đặt, xem mỗi ràng buộc là một cạnh có hướng giữa hai chữ cái và kiểm tra đồ thị có chu trình hay không. Độ phức tạp là

$$O(26^2 m + 26 \cdot \text{tổng độ dài các xâu}).$$

5.3.2 KMP

Trong thi tin học thường gặp bài toán khớp một xâu mẫu với một xâu chính. Cách làm vét cạn bắt đầu so từ ký tự đầu tiên; nếu bằng nhau thì so tiếp, đến khi gặp ký tự khác nhau. Lúc đó nó đơn giản bỏ toàn bộ thông tin đã khớp và thử lại từ vị trí kế tiếp của xâu chính với ký tự đầu tiên của xâu mẫu. Việc bỏ thông tin khớp trước đó gây lãng phí lớn và làm hiệu suất thấp.

Thuật toán KMP tăng tốc quá trình khớp xâu mẫu với xâu chính. Chi tiết của KMP có thể xem trong luận văn đội tuyển quốc gia năm 2004 của Zhu Zeyuan. Dưới đây nêu định nghĩa mảng thất bại của xâu mẫu.

Định nghĩa 3.1 (mảng thất bại). Cho xâu mẫu $s = s_1 s_2 \dots s_n$ có độ dài n . Đặt

$$\text{next}[i] = |\text{LBorder}(s[1 : i])|.$$

Mảng next được gọi là mảng thất bại của s .

Ý tưởng cốt lõi của KMP là tiền xử lý mảng thất bại trên xâu mẫu, rồi với xâu chính, dựa vào mảng này để tính với mỗi vị trí kết thúc thì có thể khớp tối đa bao nhiêu ký tự của xâu mẫu. Thời gian của KMP là tuyến tính.

KMP automaton. Sau khi có mảng next, nếu nối cạnh từ i tới $\text{next}[i]$ thì toàn bộ cấu trúc tạo thành một cây $n + 1$ đỉnh. Trong đó đỉnh 0 là gốc, đỉnh i biểu diễn $\text{pre}[i]$. Các đỉnh trên đường từ gốc tới đỉnh i chính là các Border của $s[1 : i]$. Vì vậy có thể dựa vào cây này để xây mảng chuyển cho mỗi đỉnh, cho biết nếu sau xâu hiện tại thêm một ký tự thì xâu mới có thể khớp nhiều nhất bao nhiêu ký tự của mẫu. Khi khớp xâu, ta dùng mảng chuyển để tính nhanh. Cấu trúc này được gọi là KMP automaton.

Ví dụ 2 (NOI 2014, Zoo). Có T bộ dữ liệu. Mỗi bộ cho một xâu mẫu $s = s_1 s_2 \dots s_n$. Với mỗi $\text{pre}[i]$, cần đếm số Border không rỗng B thỏa

$$2|B| \leq i.$$

Gọi số đó là $\text{num}[i]$, yêu cầu tính mảng num .

Dữ liệu:

$$1 < T < 5, \quad 1 < n < 1000000.$$

Giới hạn thời gian là 1 giây.

Một cách giải là xây mảng next , rồi từ đó xây cây trong KMP automaton. Với mỗi đỉnh, ta chỉ cần biết trên đường từ gốc tới đỉnh đó có bao nhiêu đỉnh có nhãn không vượt quá một nửa nhãn hiện tại. Vì nhãn trên đường đi tăng đơn điệu, có thể nhị phân trực tiếp. Mảng next được tính trong thời gian tuyến tính, nên tổng độ phức tạp là $O(Tn \log n)$.

5.3.3 AC automaton

AC automaton có thể xem là cấu trúc kết hợp trie và KMP automaton. Để xây AC automaton cho nhiều xâu mẫu, trước hết xây trie, rồi giống KMP automaton, xây mảng next để tạo cây và mảng chuyển. Tuy nhiên mảng next này dành cho nhiều xâu, khác với mảng next của KMP automaton. Trong AC automaton, cây tạo bởi các cạnh kiểu này được gọi là *fail tree*. Trên fail tree, một đỉnh biểu diễn một xâu, còn các đỉnh trong cây con của nó biểu diễn những xâu có xâu đó làm hậu tố. Dựa vào fail tree ta xây được mảng chuyển của từng đỉnh và khớp nhanh khi quét xâu.

Tính chất và cách xây AC automaton có thể xem trong luận văn đội tuyển quốc gia năm 2004 của Zhu Zeyuan. Nhờ kết hợp tính chất của trie và KMP automaton, AC automaton giải được các bài toán khớp nhiều mẫu phức tạp hơn.

Ví dụ 3 (COCI 2015, Divljak). Ban đầu cho n xâu $s_1, s_2, \dots, s_n$. Có một tập xâu T , ban đầu rỗng. Có q thao tác thuộc hai loại:

1. thêm một xâu p vào tập T ;
2. chọn một xâu s_i , hỏi trong T có bao nhiêu xâu chứa s_i làm xâu con.

Đặt sum là tổng độ dài các xâu ban đầu và

$$\text{sum}_p = \sum |p|$$

trên các xâu được thêm.

Dữ liệu:

$$1 < n, q < 100000, \quad 1 < \text{sum}, \text{sum}_p < 2000000.$$

Giới hạn thời gian là 1 giây.

Trước hết xây AC automaton cho $s_1, s_2, \dots, s_n$, thu được fail tree và mảng chuyển. Giả sử trên fail tree, đỉnh u_i biểu diễn xâu s_i .

Ta chuyển đổi thao tác như sau. Với thao tác thêm xâu p , xem như tô cùng một màu cho các đỉnh của AC automaton biểu diễn các tiền tố của p . Với thao tác hỏi xâu s_i , đáp án là số màu khác nhau trong cây con của u_i trên fail tree. Chuyển đổi này suy ra trực tiếp từ tính chất của AC automaton.

Khi cài đặt, dùng thứ tự DFS của fail tree để thống kê. Mỗi cây con tương ứng một đoạn liên tiếp trên thứ tự DFS. Sau mỗi lần tô màu, nếu một cây con có điểm mang màu đó, tổng trọng số trên đoạn DFS tương ứng cần tăng một. Với thao tác hỏi, lấy đoạn DFS của cây con u_i rồi dùng Fenwick tree để lấy tổng đoạn.

Với thao tác thêm, lấy ra các đỉnh cần tô; số lượng là $O(|p|)$. Khử trùng, sắp theo thứ tự DFS, sau đó cộng 1 tại mỗi đỉnh và trừ 1 tại LCA của hai đỉnh kề nhau trong danh sách. Sau thao tác này, trong mỗi

cây con, nếu có đỉnh được tô thì chỉ đỉnh có thứ tự DFS nhỏ nhất đóng góp 1. Có thể hiểu cách làm này qua hình dạng của virtual tree.

Nhờ đó bài toán giải được trong

$$O(26\text{sum} + \text{sum}_p \log \text{sum} + \text{sum} \log \text{sum})$$

thời gian.

5.4 Các cách giải theo hậu tố

Trong các phương pháp khớp chuỗi, nhóm hậu tố giữ vị trí quan trọng. So với nhóm tiền tố, các cấu trúc hậu tố thường cao cấp và phức tạp hơn, đồng thời giải được các bài toán khó hơn. Mục này giới thiệu bốn cấu trúc dữ liệu hậu tố: suffix array, suffix tree, suffix cactus và suffix automaton.

5.4.1 Suffix array

Suffix array là một cấu trúc dữ liệu rất tốt trong xử lý chuỗi, được ứng dụng rộng rãi ở nhiều loại bài toán.

Ý tưởng cốt lõi là với xâu mẫu b , sắp xếp mọi hậu tố của b theo thứ tự từ điển và ghi bằng mảng sa . Trong đó sa_i là vị trí của hậu tố đứng thứ i theo thứ tự từ điển. Tương ứng có mảng $rank$, trong đó $rank_i$ là vị trí của hậu tố $\text{suf}[i]$ trong mảng sa . Sau khi có sa và $rank$, ta dùng tính chất của suffix array để tính mảng $height$ trong thời gian tuyến tính; $height_i$ là độ dài LCP của hai hậu tố sa_{i-1} và sa_i . Mảng $height$ giúp tính hiệu quả nhiều bài toán, ví dụ LCP của hai hậu tố.

Suffix array có thể xây bằng thuật toán nhân đôi hoặc DC3. Do khuôn khổ bài viết, phần này không trình bày chi tiết tính chất và cách xây suffix array; độc giả quan tâm có thể xem luận văn đội tuyển quốc gia năm 2004 của Xu Zhilei hoặc luận văn năm 2009 của Luo Suiqian.

Bài toán 1: LCP. Cho xâu b độ dài n . Có q truy vấn; mỗi truy vấn cho hai xâu con của b , hỏi độ dài tiền tố chung dài nhất của chúng.

Trước hết xây suffix array của b trong $O(n \log n)$, thu được sa , $rank$ và $height$. Với mỗi truy vấn LCP của hai xâu con, kéo chúng thành hai hậu tố, lấy LCP của hai hậu tố rồi lấy min với độ dài hai xâu con. LCP của hai hậu tố trở thành truy vấn RMQ trên mảng $height$. Tiền xử lý sparse table trong $O(n \log n)$, mỗi truy vấn trả lời trong $O(1)$. Tổng độ phức tạp là $O(n \log n + q)$.

Bài toán 2: ghép xâu con. Cho xâu b độ dài n . Có q truy vấn; mỗi truy vấn chọn hai xâu con a và c của b , đặt $s = ac$, hỏi s có phải xâu con của b không. Nếu có, cần đưa ra một vị trí hợp lệ.

Xây suffix array của b trong $O(n \log n)$. Nếu s là xâu con của b , thì s là tiền tố của một hậu tố nào đó của b . Trước hết tìm toàn bộ hậu tố có tiền tố a ; theo định nghĩa suffix array, các hậu tố này tạo thành một đoạn liên tiếp trên sa , tìm được bằng nhị phân. Trên đoạn đó, tiếp tục nhị phân theo thứ tự từ điển để tìm hậu tố có tiền tố ac . Trong quá trình nhị phân, dùng mảng $rank$ và phép tính LCP $O(1)$ từ bài toán 1 để so sánh nhanh. Vì vậy mỗi truy vấn mất $O(\log n)$, tổng độ phức tạp là $O(n \log n + q \log n)$.

Bài toán 3: tiền tố lớn nhất trên đoạn. Cho xâu b độ dài n . Có q truy vấn; mỗi truy vấn cho một đoạn $[l, r]$, yêu cầu tính

$$\max_{l \leq i \leq r} \text{LCP}(\text{suf}[i], b).$$

Xây suffix array của b trong $O(n \log n)$, thu được sa , $rank$ và $height$. Bài toán trở thành RMQ trên các giá trị $\text{LCP}(\text{suf}[i], b)$. Tương tự bài toán 1, tiền xử lý sparse table trong $O(n \log n)$, mỗi truy vấn trả lời $O(1)$. Tổng độ phức tạp là $O(n \log n + q)$.

Một mở rộng là vừa tính giá trị lớn nhất vừa đếm số vị trí đạt giá trị đó. Khi đó không dùng trực tiếp sparse table kiểu chỉ lấy min/max đơn giản nữa; có thể dùng nhân đôi để trả lời mỗi truy vấn trong $O(\log n)$. Độ phức tạp trở thành $O(n \log n + q \log n)$.

5.4.2 Suffix tree

Suffix tree là cấu trúc dữ liệu kinh điển cho xử lý chuỗi. Bản chất của suffix tree là trie. Với một chuỗi b độ dài n , nếu đưa tất cả hậu tố của nó vào trie thì thu được suffix tree của b . Tuy nhiên trie trực tiếp có thể có $O(n^2)$ đỉnh, nên cần nén các đường đi. Ban đầu mỗi cạnh của trie biểu diễn một ký tự; sau khi nén, mỗi cạnh biểu diễn một chuỗi. Nếu thêm ký tự kết thúc vào cuối b , xây trie của mọi hậu tố rồi gộp các đỉnh chỉ có một con với con của nó, ta thu được cây có $O(n)$ đỉnh.

Phân tích tính chất và cách xây suffix tree có thể xem trong luận văn đội tuyển quốc gia năm 2004 của Zhu Zeyuan.

Sau khi xây suffix tree, có thể thấy mảng sa ở mục trước chính là thứ tự DFS của các lá trong suffix tree. Vì vậy có thể dùng suffix tree để xây suffix array. Ngược lại, mục sau giới thiệu một cách xây suffix tree từ suffix array. Hai cấu trúc này có thể chuyển đổi qua lại; các bài toán suffix array giải được thì suffix tree cũng hỗ trợ được. Vì thế phần này không phân tích ví dụ riêng.

5.4.3 Suffix cactus

Suffix cactus, giống suffix array, cũng dựa trên suffix tree. Nó cũng là cấu trúc dạng cây tương tự suffix tree. Trong suffix tree, ta gộp các đỉnh chỉ có một con với con của chúng; còn trong suffix cactus, ta gộp mỗi đỉnh không phải lá với một con của nó, tạo thành một khối liên thông gọi là một nhánh.

Để xây suffix cactus, trước hết xây suffix array. Dựa vào định nghĩa của sa và height, khi duyệt sa tăng dần, duy trì một ngăn xếp đơn điệu giảm của mảng height. Mỗi lần duyệt tới vị trí i , nối sa_i với phần tử hiện ở đỉnh ngăn xếp.

Do khuôn khổ bài viết, tính đúng đắn của cách xây dựng trên không được trình bày chi tiết. Tính chất cụ thể của suffix cactus có thể xem trong tài liệu giao lưu trại đông năm 2014 của Wang Yuetong, Xu Duo và Xu Zihan.

Từ suffix array có thể thu được suffix cactus. Nếu cắt mỗi nhánh của suffix cactus tại các vị trí nối cạnh, ta nhận được suffix tree. Vì vậy chỉ cần ghi lại vị trí nối cạnh khi dựng suffix cactus, ta có thể xây suffix tree từ suffix array trong thời gian tuyến tính.

5.4.4 Suffix automaton

Cây của suffix automaton là suffix tree của chuỗi mẫu đảo ngược. Vì suffix tree là một trie nén, suffix automaton cũng hỗ trợ duy trì nhiều chuỗi mẫu. Do đó ta có một cách khác để xây suffix tree của một chuỗi mẫu: trước hết xây suffix automaton, rồi xây suffix tree từ suffix automaton. Suffix automaton có thể được xây bằng phương pháp tăng dần. Do khuôn khổ bài viết, cách xây và phân tích tính chất có thể xem trong tài liệu giao lưu trại đông năm 2012 của Chen Lijie.

Ví dụ 7 (ZJOI 2015, Vùng đất huyền tường được chư thần ưu ái). Cho một cây n đỉnh, mỗi đỉnh mang một ký tự. Mỗi lần chọn hai đỉnh x, y , ghép các ký tự trên đường đi từ x tới y theo thứ tự để được một chuỗi. Hỏi tất cả các đường đi như vậy tạo ra bao nhiêu chuỗi khác nhau. Số lá của cây không vượt quá 20, kích thước bảng chữ cái không vượt quá 10.

Dữ liệu:

$$1 < n < 100000.$$

Giới hạn thời gian là 1 giây.

Cách vét cạn: vì chỉ có 20 lá, lấy xâu trên đường đi giữa mọi cặp lá. Mọi xâu con của các xâu giữa lá đều tương ứng với một đường đi trên cây, và mỗi đường đi đều được một xâu con như vậy biểu diễn. Do đó có thể xây suffix automaton cho 400 xâu này rồi đếm số xâu con khác nhau trên automaton. Mỗi xâu có độ dài $O(n)$, nên độ phức tạp là $O(20^2 n \cdot 10)$.

Cách tốt hơn là xét từng lá làm điểm xuất phát. Nếu lấy một lá làm gốc, toàn bộ xâu từ lá đó tới các lá khác có thể hiểu là duyệt qua cả cây một lần. Vì vậy tổng độ dài xâu khi cố định một lá làm gốc là $O(n)$. Độ phức tạp trở thành $O(20n \cdot 10)$.

5.5 Border Tree

Mục này giới thiệu cấu trúc dữ liệu Border Tree. Cấu trúc này dựa trên KMP automaton và xử lý hiệu quả một lớp bài toán khớp một xâu mẫu.

Xét xâu mẫu b độ dài m . Nếu xây KMP automaton cho b , chiều sâu của cây trong automaton có thể đạt $O(m)$. Border Tree là một biến thể của cây đó: nó gộp một số tiền tố cùng loại, làm chiều sâu giảm xuống $O(\log m)$. Nói cách khác, mỗi đỉnh của Border Tree không còn biểu diễn một tiền tố duy nhất mà biểu diễn một nhóm tiền tố; nhóm đó gọi là *Border Group*.

5.5.1 Border Group

Số Border của $b[1 : i]$ có thể đạt $O(i)$, chẳng hạn $b[1 : i] = 11 \dots 1$. Ta cần nén lượng thông tin này. Ý tưởng cơ bản là phân loại Border sao cho số loại Border của $b[1 : i]$ chỉ còn $O(\log i)$.

Định nghĩa 5.1 (Border Group). Các Border của $b[1 : i]$ được chia thành một dãy

$$g = [B_1, B_2, \dots, B_{|g|}], \quad 1 \leq |g| \leq O(\log i).$$

Mỗi B_j là một dãy các Border, và mọi Border trong B_j dài hơn mọi Border trong B_{j+1} .

Mỗi B_j có một trong hai dạng:

$$[x] \quad (k_j = 2),$$

hoặc

$$[x^k r, x^{k-1} r, \dots, x r] \quad (k_j > 1),$$

trong đó $x^k r$ là một Border của $b[1 : i]$, còn r là một tiền tố của x . Ta gọi mỗi B_j là một Border Group, và g là dãy BG.

Ví dụ 8. Cho

$$b = \text{bababbababcbababbabab}.$$

Các Border là

$$[\text{bababbabab}, \text{babab}, \text{bab}, \text{b}].$$

Có thể chia chúng thành ba nhóm:

$$[\text{bababbabab}], \quad [\text{babab}, \text{bab}], \quad [\text{b}].$$

Nhóm thứ nhất có thể viết với $r = \text{babab}, x = \varepsilon$; nhóm thứ hai với $r = \text{ab}, x = \text{b}$; nhóm thứ ba với $r = \text{b}, x = \varepsilon$.

Dãy BG được xây theo ý tưởng: dùng Border Group để phân phối mọi Border vào các dãy khác nhau; trong cùng một Border Group, các Border có cùng x và r . Quy trình:

1. Lấy $a = \text{LBorder}_i$, phân tích $a = x^k r$, rồi đưa $x^k r, x^{k-1} r, \dots, x r$ vào B_1 .
2. Đặt $s = x r$. Nếu s là xâu tuần hoàn thì lấy LBorder của s làm phần tử đầu của B_2 ; nếu không thì lấy chính s làm phần tử đầu của B_2 .
3. Lặp lại cho đến khi cả x và LBorder đều rỗng.

Cách xây này bảo đảm $1 \leq |g| \leq O(\log m)$. Lý do là ở bước 2, nếu s tuần hoàn thì s là phần tử đầu của Border Group kế tiếp. Theo định nghĩa, $|r| < |x|$, nên $|s| < \frac{k+1}{k}|x|$; trường hợp lớn nhất khi $k = 2$, do đó với mọi k đều có $|s| < \frac{3}{2}|x|$. Nếu s không tuần hoàn, phần tử đầu của nhóm kế tiếp là $\text{LBorder}(s)$. Theo định nghĩa LBorder ,

$$2|\text{LBorder}(s)| < |s|.$$

Vì $|s| = |x r|$, độ dài phần tử đầu giảm theo hằng số, nên số nhóm là $O(\log m)$.

5.5.2 Xây Border Tree

Để xây Border Tree, trước hết xây KMP automaton, sau đó chuyển cây trong automaton thành Border Tree.

Với mỗi đỉnh i của KMP automaton, định nghĩa bộ ba (x_i, y_i, z_i) :

1. Nếu $b[1 : i]$ là xâu tuần hoàn và $b[1 : i] = x^k r$ với $k > 1$, đặt $(x_i, y_i, z_i) = (|r|, |x|, k)$.
2. Nếu $b[1 : i]$ không tuần hoàn và $b[1 : i] = x r$, đặt $(x_i, y_i, z_i) = (|r|, |x|, 1)$.
3. Nếu $b[1 : i]$ không tuần hoàn nhưng có đúng một con j thỏa $z_j = 2$, đặt $(x_i, y_i, z_i) = (x_j, y_j, 1)$.

Trong trường hợp 3, định lý sau bảo đảm nhiều nhất chỉ có một con j thỏa $z_j = 2$.

Định lý 5.1. Mỗi xâu không tuần hoàn $s = b[1 : i]$ không thể có từ hai con trở lên là xâu tuần hoàn.

Chứng minh. Giả sử ngược lại s có ít nhất hai con là xâu tuần hoàn. Khi đó dựa trên hai con tuần hoàn đó, có thể biểu diễn

$$s = x^k r = y^\ell t,$$

với $|x| < |y|$ và $|y| \bmod |x| \neq 0$. Nếu $|y| \bmod |x| = 0$, thì s cũng có dạng tuần hoàn, mâu thuẫn giả thiết.

Đặt $d = \gcd(|x|, |y|)$. Khi đó s có một cách biểu diễn với chu kỳ độ dài d . Điều này dẫn đến $|x| \bmod d = 0$, mâu thuẫn với điều kiện đã chọn. Vậy giả thiết sai.

Bộ ba (x_i, y_i, z_i) có thể tính trong thời gian tuyến tính bằng hai lượt duyệt:

1. Lượt duyệt thứ nhất xử lý các đỉnh thuộc trường hợp 1 và 2 theo định nghĩa Border Group:

$$x_i = |r|, \quad y_i = |x|, \quad z_i = \left\lfloor \frac{i}{|x|} \right\rfloor.$$

2. Lượt duyệt thứ hai xử lý trường hợp 3 và cập nhật (x_i, y_i, z_i) .

Trường hợp 3 có ý nghĩa duy trì Border Group dạng $x^k r$. Ví dụ, với

$$b = abaaba,$$

ban đầu $(x_3, y_3, z_3) = (1, 2, 1)$, nhưng vì $(x_6, y_6, z_6) = (0, 3, 2)$, nên (x_3, y_3, z_3) được đổi thành $(0, 3, 1)$.

Sau khi mỗi đỉnh trong KMP automaton có một bộ ba, ta sửa cây của automaton để xây Border Tree. Những đỉnh có cùng x, y thuộc cùng một loại theo định nghĩa Border Group; chỉ giữ lại đỉnh có z nhỏ nhất và ghi thông tin của các đỉnh còn lại vào đó. Chẳng hạn nếu trong một loại có đỉnh biểu diễn r thì giữ đỉnh đó, nếu không thì giữ đỉnh biểu diễn $x r$. Cuối cùng duyệt thêm một lần để thu được Border Tree.

5.5.3 Phân tích tính chất

Mỗi đỉnh i trong Border Tree tương ứng với một Border Group, có thể viết dưới dạng

$$[x_i^{z_i} r_i, x_i^{z_i-1} r_i, \dots, x_i r_i].$$

Tuy nhiên, với một đỉnh i , các Border Group trên đường từ gốc tới i không nhất thiết biểu diễn chính xác mọi Border của $b[1 : i]$. Có thể có nhóm chứa cả những xâu không phải Border thật của $b[1 : i]$. Do đó khi cần tìm mọi Border của $b[1 : i]$, ta phải duy trì thêm một cận độ dài x trong quá trình duyệt lên tổ tiên để lọc các phần tử hợp lệ.

Ký hiệu $\text{fa}[i]$ là cha của i trong Border Tree. Trong Border Group của $\text{fa}[i]$, các Border thật của $b[1 : i]$ có dạng

$$x_j^t r_j$$

với số mũ t nằm trong một đoạn liên tiếp; biên trên của đoạn được lấy từ bộ ba tương ứng với $\text{next}[i]$ trong KMP automaton. Vì vậy khi duyệt các Border Group trên đường đi, ta chỉ cần cập nhật cận này.

5.5.4 Ví dụ

Cho

$$b = \text{babababcbabab},$$

độ dài 13. Trong bản gốc, Hình 1 là cây của KMP automaton và Hình 2 là Border Tree tương ứng. Cây ban đầu có 14 đỉnh, chiều sâu 5. Sau khi phân loại Border và xây Border Tree, cây còn 10 đỉnh, chiều sâu 3. Các đỉnh hình vuông trong cây KMP automaton được co lại khi xây Border Tree.

Bảng trong bản gốc so sánh trạng thái của từng đỉnh trong KMP automaton với bộ ba (x_i, y_i, z_i) của Border Tree. Ý nghĩa chính là: các đỉnh biểu diễn các Border cùng dạng $x^z r$ được gộp theo cùng x, y , chỉ giữ thông tin cần thiết để khi truy vấn có thể sinh lại các Border hợp lệ.

5.6 Một lớp bài toán khớp văn bản

Như đã biết, khớp xâu văn bản động với xâu mẫu tĩnh là một bài toán kinh điển. Mục này đưa ra một cách giải tổng quát hiệu quả dựa trên Border Tree. Nội dung về Border Tree đã được trình bày ở mục trước.

5.6.1 Các bài toán con

Xét xâu mẫu b độ dài m . Trước khi xử lý lớp bài toán này, cần giải ba bài toán con trên b :

1. **LCP.** Trên b , cho hai xâu con, hỏi độ dài tiền tố chung dài nhất. Cách giải xem ở Ví dụ 4. Tiền xử lý $O(m \log m)$, mỗi truy vấn $O(1)$.
2. **Ghép xâu con.** Trên b , cho hai xâu con b' và b'' , đặt $s = b' b''$, hỏi s có phải xâu con của b hay không; nếu có, đưa ra một vị trí hợp lệ. Cách giải xem ở Ví dụ 5. Tiền xử lý $O(m \log m)$, mỗi truy vấn $O(\log m)$.
3. **Tiền tố lớn nhất trên đoạn.** Trên b , cho một đoạn $[l, r]$, tính

$$\max_{l \leq i \leq r} \text{LCP}(\text{suf}[i], b).$$

Cách giải xem ở Ví dụ 6. Tiền xử lý $O(m \log m)$, mỗi truy vấn $O(1)$. Nếu còn cần đếm số vị trí đạt cực đại, mỗi truy vấn mất $O(\log m)$.

5.6.2 Chuyển đổi mô hình

Trước hết đơn giản hóa bài toán khớp văn bản động với mẫu tĩnh thành bài toán cơ sở: cho xâu văn bản a và xâu mẫu b , sau mỗi lần sửa a , cần khớp a với b . Thao tác sửa có thể là đổi một ký tự, chèn một ký tự hoặc xóa một ký tự trong a .

Chuyển bài toán khớp a với b thành tính hàm sau. Định nghĩa

$$F(a, b) = (x, y),$$

trong đó

$$x = \max_{1 \leq i \leq |a|} \text{LCP}(a[i : |a|], b),$$

còn y là số vị trí i đạt giá trị lớn nhất đó. Bài toán trở thành: khi a thay đổi động và b tĩnh, duy trì $F(a, b)$. Trong phần sau, a là xâu văn bản và b là xâu mẫu.

5.6.3 Phủ

Mục này đưa ra khái niệm *phủ*. Giả sử xâu mẫu b có độ dài m , xâu văn bản a có độ dài n . Ý tưởng cơ bản là dùng các xâu con của b để phủ chính xác a .

Định nghĩa 6.1. Một phủ S của a theo b là một dãy xâu

$$[v_1, v_2, \dots, v_t],$$

trong đó mỗi v_i là một phần tử phủ, thỏa:

$$v_1 v_2 \cdots v_t = a,$$

và với mọi $1 \leq i \leq t$:

- *tính chất xâu con*: v_i là xâu con của b ;
- *tính chất cực đại*: nếu $i < t$, thì $v_i v_{i+1}$ không phải xâu con của b .

Đặc biệt, nếu một ký tự của a không xuất hiện trong b , ta cũng xem ký tự đó là một v_i , nhưng trong b nó tương ứng với xâu rỗng.

Một xâu có thể có nhiều cách phủ hợp lệ; khi cài đặt chỉ cần chọn một cách bất kỳ. Có thể hiểu đơn giản là dãy v_i chia a thành các khối nhỏ. Với mỗi v_i , dùng bộ ba (s_i, l_i, r_i) để biểu diễn:

- nếu v_i tương ứng với xâu con của b , thì $v_i = b[l_i : r_i]$;
- nếu v_i là ký tự không xuất hiện trong b , đặt $l_i = r_i = 0$;
- s_i là vị trí của v_i trong a , tức $s_i = 1 + \sum_{j < i} |v_j|$.

Khởi tạo phủ S như sau:

1. Tạo mảng c theo bảng chữ cái. Với ký tự i , nếu nó xuất hiện ở vị trí x trong b , đặt $c_i = x$, nếu không đặt $c_i = 0$.
2. Duyệt từng ký tự a_i . Nếu $c_{a_i} = 0$, ký tự đó không xuất hiện trong b ; nếu không, tạo $v_i = (i, c_{a_i}, c_{a_i})$.
3. Cách dựng này chưa thỏa tính chất cực đại. Duyệt theo thứ tự tăng của s_i , lần lượt kiểm tra $v_i v_{i+1}$ có phải xâu con của b hay không; nếu có thì gộp, nếu không thì giữ nguyên. Phép kiểm tra dùng bài toán con 2.

Sau khi có phủ ban đầu, xét cách duy trì động. Dưới đây chỉ mô tả thao tác đổi một ký tự; chèn và xóa tương tự. Khi đổi vị trí x thành ký tự y :

1. Tìm phần tử v_i đang phủ vị trí x . Vì phủ là chính xác, phần tử này là duy nhất.
2. Tách v_i thành ba phần trái, giữa, phải:

$$v_l = (s_i, l_i, l_i + x - s_i - 1),$$

$$v_m = (x, 0, 0),$$

$$v_r = (x + 1, l_i + x + 1 - s_i, r_i).$$

Phần giữa đúng là vị trí x . Theo ký tự mới, đặt nó thành phần tử tương ứng với y , rồi xóa các phần có độ dài 0 trong a .

3. Do tính chất cực đại, sau khi tách chỉ các phần kề v_l và v_r mới có khả năng thay đổi; đưa chúng vào danh sách cần xét.
4. Lúc này chỉ có hằng số phần tử có thể chưa cực đại. Duyệt theo thứ tự tăng của s_i , dùng bài toán con 2 để kiểm tra hai phần kề nhau có gộp được không, từ đó tái dựng các phần tử cho thỏa tính chất cực đại.

Ví dụ 10. Cho

$$a = \text{cbabbabbbacb}, \quad b = \text{abbacb}.$$

Phủ ban đầu là

$$[\text{cb}, \text{abba}, \text{bb}, \text{bacb}].$$

Nếu đổi ký tự thứ 4 của a thành c , phần tử phủ vị trí này là abba . Tách nó thành a , b , ba , đổi phần giữa thành c , rồi gộp được a với c . Phủ mới là

$$[\text{cb}, \text{ac}, \text{ba}, \text{bb}, \text{bacb}],$$

thỏa định nghĩa.

Phân tích độ phức tạp. Bài toán con 2 cần tiền xử lý $O(m \log m)$, mỗi truy vấn $O(\log m)$. Khi khởi tạo, ta thực hiện $O(n)$ lần kiểm tra kiểu bài toán con 2, nên tốn $O(n \log m)$. Với q lần sửa, mỗi lần duy trì phủ chỉ cần hằng số lần kiểm tra, tốn $O(q \log m)$.

Cần một cấu trúc dữ liệu hỗ trợ chèn, xóa, tìm tiền nhiệm, hậu nhiệm và tìm vị trí đầu tiên có chỉ số không nhỏ hơn một giá trị cho trước. Nếu dùng cây tìm kiếm nhị phân để duy trì phủ S , mỗi thao tác tốn $O(\log n)$. Vì mỗi lần sửa chỉ thực hiện hằng số thao tác trên cây, thời gian duy trì phủ là

$$O(q \log n + q \log m).$$

Kết hợp lại, bài toán duy trì phủ giải được trong

$$O((n + m + q) \log m + q \log n).$$

Nếu ghi trực tiếp s_i làm chỉ số, có thể dùng cây đoạn thay cây tìm kiếm nhị phân; độ phức tạp không đổi nhưng hằng số nhỏ hơn. Tốt hơn nữa, dùng van Emde Boas tree thay cây tìm kiếm nhị phân thì thao tác trên cấu trúc thứ tự có thể giảm xuống $O(\log \log n)$.

5.6.4 Kỹ thuật tính toán

Mục trước đã cho cách duy trì phủ S hiệu quả. Mục này đơn giản hóa bài toán để giới thiệu kỹ thuật tính $F(a, b)$ dựa trên Border Tree và phủ S .

Bài toán đơn giản hóa. Bản chất của $F(a, b)$ là biểu diễn giá trị lớn nhất của

$$\text{LCP}(a[i : |a|], b) \quad (1 \leq i \leq |a|)$$

và số vị trí đạt giá trị đó thành một cặp. Phủ S chia a thành các khối $v_1, \dots, v_t$, trong đó $v_i = b[l_i : r_i]$ nếu nó xuất hiện trong b . Với mỗi khối, đặt w_i là cặp gồm giá trị lớn nhất và số lần đạt giá trị lớn nhất của

$$\text{LCP}(v_i[j : |v_i|] \cdot v_{i+1} \cdots v_t, b)$$

trên các vị trí j trong v_i . Khi đã biết các w_i , ta gộp chúng để thu được $F(a, b)$.

Do tính chất cực đại của phủ, giá trị w_i không đổi nếu phần đuôi $v_{i+1} \cdots v_t$ được rút gọn theo cùng thông tin biên cần thiết. Mục trước đã cho cách tìm tiền nhiệm và hậu nhiệm của một phần tử phủ trong $O(\log n)$, nên mỗi lần cần tính w_i , ta chỉ mất $O(\log n)$ để tìm các phần tử kề.

Vì vậy bài toán tính $F(a, b)$ được rút về tính từng w_i . Với w_i , ta xét xâu

$$s = s_1 s_2 s_3,$$

trong đó s_1 là một xâu con của b , còn $s_2 s_3$ là phần được ghép từ các phần tử kế tiếp. Cần gộp giá trị lớn nhất và số lần đạt của

$$\text{LCP}(s[i : |s|], b) \quad (1 \leq i \leq |s_1|).$$

Bài toán mới. Đặt $s = s_1 s'$ và $s_1 = b[L : R]$. Cần tính giá trị lớn nhất Max và số lần đạt Num của

$$\text{LCP}(s[i : |s|], b) \quad (1 \leq i \leq |s_1|).$$

Với mỗi i , đặt

$$j = i + \text{LCP}(s[i : |s|], b) - 1.$$

Khi đó $s[i : j]$ là một tiền tố của b . Nếu s_1 chỉ là một ký tự và ký tự đó không xuất hiện trong b , thì $\text{Max} = 0, \text{Num} = 1$. Trường hợp còn lại chia theo $j < |s_1|$ và $j \geq |s_1|$.

Trường hợp 1: $j < |s_1|$. Khi $j < |s_1|$, bài toán chuyển thành

$$\max \text{LCP}(s_1[i : |s_1|], b).$$

Vì $s_1 = b[L : R]$, các hậu tố $s_1[i : |s_1|]$ tương ứng với đoạn con trong b . Nếu không có ràng buộc $j < |s_1|$, ta có thể dùng trực tiếp bài toán con 3. Ràng buộc này yêu cầu vị trí khớp xa nhất trong b phải nằm nghiêm ngặt trước R .

Cách đơn giản là khi tính bài toán con 3, chỉ xét các hậu tố có vị trí khớp xa nhất trong b nhỏ hơn R . Xét mọi Border của $b[L : R]$, tìm x lớn nhất sao cho $b[1 : x]$ là Border của $b[L : R]$ và

$$0 < x < R - L + 1.$$

Khi đó chỉ cần làm bài toán con 3 trên đoạn $[L, R - x]$; đoạn còn lại $[R - x + 1, R]$ cho đáp án nhỏ hơn x nên có thể bỏ qua.

Để tối đa hóa x , có thể xây KMP automaton rồi từ đỉnh R nhảy lên gốc bằng nhân đôi. Cũng có thể dùng Border Tree: vì chiều sâu Border Tree là $O(\log m)$, duyệt trực tiếp các Border Group trên đường lên gốc vẫn bảo đảm độ phức tạp.

Ví dụ 11. Cho

$$a = \text{babaca}, \quad b = \text{abcabacabc}.$$

Phủ hiện tại là

$$[b, \text{abaca}],$$

trong đó $v_1 = b$, $v_2 = \text{abaca}$, và v_2 tương ứng với bộ ba $(2, 4, 8)$ trong b . Khi cần tính w_2 , ta có $\text{LCP}(v_2, b) = 2$, tiền tố tương ứng là ab , thuộc trường hợp 1.

Xét đoạn $[4, 8]$ trong b . Nếu trực tiếp làm bài toán con 3 trên $[4, 8]$, đáp án sai sẽ là 3, ứng với tiền tố abc và đoạn khớp $[8, 10]$. Cần tìm x như trên; ở ví dụ này $x = 1$. Làm bài toán con 3 trên $[4, 8 - x]$ thu được đáp án 2, tiền tố ab , đoạn khớp $[3, 5]$, vị trí xa nhất nhỏ hơn 8. Vì vậy với w_2 ,

$$\text{Max} = 2, \quad \text{Num} = 1.$$

Trường hợp 2: $j \geq |s_1|$. Vì s_1 là xâu con của b , tìm một đoạn $[L, R]$ sao cho

$$s_1 = b[L : R].$$

Nếu $j \geq |s_1|$, thì phần khớp vượt qua cuối s_1 , nên

$$s_1[i : |s_1|]$$

phải là một Border của $b[L : R]$. Một cách vét cạn là xây KMP automaton, duyệt mọi Border của $b[L : R]$. Với mỗi Border có độ dài hợp lệ, ghép Border đó với phần s' ở sau để được xâu mới rồi tính LCP với b . Vì xâu mới được ghép từ ba xâu con của b , chỉ cần dùng hằng số lần bài toán con 1 để tính được giá trị; khi đó $\text{Num} = 1$.

Nhược điểm là chiều sâu KMP automaton có thể đạt $O(m)$, nên vét cạn mỗi lần có thể tốn $O(m)$. Dùng Border Tree để tối ưu: nếu xử lý được cả một Border Group trong $O(1)$, độ phức tạp mỗi lần chỉ còn $O(\log m)$.

Xét một Border Group có dạng

$$[x^k r, x^{k-1} r, \dots, x r].$$

Bản chất của chuyển đổi là thay vì xét từng Border trong KMP automaton, ta xét cả nhóm Border có cùng tính chất. Nếu Border Group đặc biệt có r rỗng, ta xem $x^0 r$ là xâu rỗng và xử lý riêng Border rỗng; phần còn lại của nhóm vẫn có cùng dạng.

Gọi phần sau s' là xâu cần ghép sau Border. Có hai loại chính:

1. Nếu s' là xâu tuần hoàn có độ dài chu kỳ $|x|$, đặt

$$\text{len} = \text{LCP}(s', b[|x| + 1 : |b|]).$$

Với một Border trong nhóm, độ dài khớp nhận được có dạng

$$\min\{|b|, |x^t r| + \text{len}\}.$$

Trực giác là khi LCP còn nằm trong phần tuần hoàn của b , các Border trong cùng nhóm chỉ khác số lần lặp x , còn phần ghép sau đóng góp cùng một len . Nếu độ dài khớp lớn nhất đạt $|b|$, thì $\text{Max} = |b|$ và Num là số giá trị t hợp lệ; nếu không, Max là độ dài khớp lớn nhất và $\text{Num} = 1$.

2. Nếu không thuộc loại trên, tồn tại vị trí sai khác đầu tiên q . Từ thông tin Border Group và một số phép tính LCP, ta xác định được Border nào cho khớp dài nhất. Việc phân tích chia thành các trường hợp theo vị trí của q trong khối tuần hoàn; mỗi trường hợp hoặc trả lời trực tiếp, hoặc rút về loại 1 trên một đoạn ngắn hơn.

Ví dụ 12. Cho

$$b = ababababa, \quad s_1 = ababab, \quad s_2 = ab, \quad s_3 = ba.$$

Khi đó $s' = abba$. Vì s_1 là xâu con của b , có thể chọn $[L, R] = [3, 8]$. Trong KMP automaton, các Border của $b[1 : 8]$ có độ dài hợp lệ là

$$ababab, \quad abab, \quad ab.$$

Ghép chúng với s' lần lượt cho các độ dài khớp 8, 6, 4, nên $\text{Max} = 8, \text{Num} = 1$.

Trong Border Tree, mọi Border của $b[1 : 8]$ thuộc cùng một Border Group, có bộ ba $(0, 2, 4)$. Vì b là xâu tuần hoàn với chu kỳ 2, đây là loại 1. Nhóm gồm ba Border $ababab, abab, ab$. Từ Border ab , tính vết cận được độ dài khớp cơ sở và $\text{len} = 2$; công thức trên cho ngay độ dài 8, 6 của hai Border còn lại.

Ví dụ 13. Cho

$$b = ababababacde, \quad s_1 = ababab, \quad s_2 = ababac, \quad s_3 = de.$$

Khi đó $s' = ababacde$, và có thể chọn $[L, R] = [3, 8]$. Trong Border Tree, mọi Border của $b[1 : 8]$ cũng thuộc Border Group có bộ ba $(0, 2, 4)$, nhưng b không còn là xâu tuần hoàn hoàn toàn, nên thuộc loại 2. Nhóm có ba Border:

$$ababab, \quad abab, \quad ab.$$

Tính theo các đại lượng trong phân tích loại 2 cho thấy Border $abab$ cho giá trị tốt nhất, vì vậy

$$\text{Max} = 12, \quad \text{Num} = 1.$$

Phân tích độ phức tạp. Theo phân tích của hai trường hợp trên, nếu dùng Border Tree, số Border Group cần duyệt là $O(\log m)$. Mỗi Border Group được xử lý bằng cách phân loại và chuyển đổi cụ thể, về mặt độ phức tạp tương đương hằng số lần gọi bài toán con 1. Vì bài toán con 1 trả lời $O(1)$, tính một w_i tốn $O(\log m)$.

Kết hợp với phân tích duy trì phủ, bài toán khớp văn bản động a với mẫu tĩnh b , tức duy trì $F(a, b)$, giải được trong

$$O((n + m + q) \log m + q \log n).$$

5.6.5 Ví dụ

Ví dụ 14 (kiểm tra lẫn nhau đội tuyển 2015, Sone2). Cho xâu chính a độ dài n và xâu mẫu b độ dài m . Có q thao tác thuộc bốn loại:

1. sửa một vị trí của a , rồi hỏi $F(a, b)$;
2. chọn một đoạn $[L, R]$ của a , hỏi $F(a[L : R], b)$;
3. bài toán con 1;
4. bài toán con 2.

Dữ liệu:

$$1 < n, m, q, \Sigma < 100000.$$

Giới hạn thời gian là 2 giây.

Trong bài này, ngoài thao tác loại 2, các phép tính khác đã được nêu ở trên. Với loại 2, ý tưởng đơn giản là duy trì phủ S' của xâu $a[L : R]$ rồi tính trên đó. Vì $a[L : R]$ là xâu con của a , bỏ hai phần tử đầu

và cuối có thể bị cắt ra thì phần còn lại của S' là một dãy con liên tiếp của phủ S . Do đó với loại 2 chỉ cần xử lý riêng hai phần tử đầu cuối, còn phần giữa lấy trực tiếp từ S bằng cấu trúc dữ liệu. Nhờ vậy có thể dựng S' trong

$$O(\log n + \log m)$$

cho mỗi truy vấn. Nếu đồng thời duy trì các giá trị w_i , có thể tính ngay đáp án từ S' . Tổng độ phức tạp vẫn là

$$O((n + m + q) \log m + q \log n).$$

Bài này còn có một phần điểm trong đó xâu mẫu là ngẫu nhiên. Khi b ngẫu nhiên, chiều sâu của KMP automaton dựng trên b là $O(\log m)$. Vì thế ở dữ liệu này không cần dùng Border Tree; có thể dùng trực tiếp KMP automaton thay thế. Độ phức tạp tiệm cận không giảm, nhưng độ khó cài đặt giảm đáng kể.

Có thể tăng cường bài toán như sau. Cho một cây n đỉnh, mỗi đỉnh có một ký tự. Với một đường đi trên cây, ghép ký tự các đỉnh theo thứ tự để được xâu a . Cho thêm xâu mẫu b độ dài m . Mỗi thao tác đổi ký tự của một đỉnh trên cây và hỏi $F(a, b)$ cho một đường đi. Tăng cường hơn nữa, cây có thể trở thành cây động.

Cách giải là dùng Link-Cut Tree để duy trì phủ S . Trong mỗi splay, duy trì phủ S' của xâu tạo bởi heavy path tương ứng. Mỗi thao tác access ghép hoặc tách các splay, chỉ làm tái dựng hằng số phần tử v_i và w_i . Vì vậy bài toán trên cây có thể giải trong

$$O((n + m) \log m + q \log n \log m).$$

5.7 Lời cảm ơn

Xin cảm ơn Hiệp hội Máy tính Trung Quốc đã cung cấp nền tảng học tập và giao lưu.

Xin cảm ơn các thầy Chen Heli, Dong Yehua, You Guanghui và Shao Hongxiang của Trường Trung học số 1 Thiệu Hưng đã nhiều năm quan tâm và hướng dẫn tôi.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Yu Linyun và Chen Xuji đã chỉ dẫn.

Xin cảm ơn các anh Yu Yili, Dong Anhua và He Qizheng của Đại học Thanh Hoa đã giúp đỡ tôi.

Xin cảm ơn các bạn Zhang Hengjie, Ren Zhizhou và Jia Yuekai của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở cho tôi.

Xin cảm ơn các bạn Yang Jiacheng, Guo Yu, Shao Jiechang, Weng Tiandong, Tao Yuanzheng, Hong Huadun và các bạn khác ở Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở cho tôi.

Xin cảm ơn bạn Jia Yuekai của Trường Trung học số 1 Thiệu Hưng đã đọc duyệt bài viết này.

Xin cảm ơn các thầy cô và bạn học khác từng giúp đỡ, gợi ý cho tôi.

Xin cảm ơn cha mẹ đã quan tâm và chăm sóc tôi.

Tài liệu tham khảo của bài viết

- [1] Wikipedia, Suffix array.
- [2] Zhu Zeyuan, *Thuật toán khớp nhiều xâu và những gợi mở của nó*.
- [3] Yang Yu, *Một lớp ứng dụng của hashing trong thi tin học*.
- [4] Luo Suiqian, *Suffix array: công cụ mạnh để xử lý chuỗi*.
- [5] Dong Huaxing, *Bàn về ứng dụng trie trong thi tin học*.
- [6] Chen Lijie, *Suffix Automaton*.

- [7] Wang Yuetong, Xu Duo and Xu Zihan, *Suffix Cactus*.
- [8] Amihood Amir, Gad M. Landau, Moshe Lewenstein and Dina Sokol, “Dynamic Text and Static Pattern Matching”.
- [9] Mokhtar S. Bazaraa, John J. Jarvis and Hanif D. Sherali, *Linear Programming and Network Flows*, 4th ed., John Wiley & Sons.
- [10] Liu Rujia and Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Tsinghua University Press.
- [11] Liu Rujia, *Nhập môn kinh điển về thi thuật toán*, Tsinghua University Press.

6 Suffix automaton và ứng dụng

Zhang Tianyang

Trường Trung học số 1 Trường Sa, Hồ Nam

Tóm tắt

Suffix automaton là một công cụ xử lý xâu mới nổi trong OI, ngày càng được nhiều người ra đề ưa chuộng. Bài viết này nhằm tổng kết các tính chất và một số ứng dụng của suffix automaton, đồng thời thông qua một số ví dụ để nghiên cứu sâu hơn các tính chất của nó.

6.1 Định nghĩa và xây dựng suffix automaton

6.1.1 Định nghĩa suffix automaton

Suffix automaton (SAM) của một xâu S là một automaton hữu hạn xác định (DFA); nó có thể, và chỉ có thể, chấp nhận mọi hậu tố của S , đồng thời có số trạng thái và số chuyển nhỏ nhất.

Ta sẽ chứng minh bên dưới rằng số trạng thái và số chuyển của suffix automaton của một xâu S đều là $O(|S|)$.

6.1.2 Cách xây dựng suffix automaton

Một vài định nghĩa. Gọi xâu mẹ của suffix automaton là S . Ký hiệu $S[l, r]$ là xâu con gồm các ký tự từ vị trí l đến vị trí r trong S . Ký hiệu hậu tố bắt đầu tại vị trí i là suf_i , và tiền tố kết thúc tại vị trí i là pre_i .

Ký hiệu SAM_{str} là trạng thái (hay nút) nhận được sau khi bắt đầu từ trạng thái đầu và đọc xâu str .

Với một xâu con str của S , ký hiệu right_{str} là tập các vị trí đầu mút phải của mọi lần xuất hiện của str trong S .

Nếu không nói khác, ta giả sử bảng chữ cái gồm mọi chữ cái thường. Kích thước bảng chữ cái được xem là hằng số và không tính vào độ phức tạp.

Chứng minh độ phức tạp. Xét một xâu str . Nếu str là một xâu con của S , thì SAM_{str} phải là một trạng thái hợp lệ. Lý do là ta có thể thêm một ký tự vào sau str để biến nó thành một hậu tố của S ; hậu tố này phải là một trạng thái chấp nhận được, nên SAM_{str} cũng phải là một trạng thái hợp lệ. Ngược lại, nếu str không phải xâu con của S , thì SAM_{str} phải là trạng thái không hợp lệ (hay trạng thái rỗng), vì dù thêm ký tự thế nào nó cũng không thể trở thành một hậu tố của S .

Nói cách khác, với mỗi xâu con của S , trong suffix automaton đều phải có một trạng thái tương ứng. Tuy nhiên rõ ràng ta không thể dựng riêng một trạng thái cho từng xâu con, vì số xâu con là $O(|S|^2)$.

Xét một xâu con str của S . Nếu $right_{str}$ và $right_{a+str}$ (trong đó a là một xâu) hoàn toàn giống nhau, thì ta có thể gộp chúng vào cùng một trạng thái. Từ hai xâu này, các ký tự có thể thêm vào để thu được hậu tố là như nhau (hay nói khác đi, ngoài phần xâu a ở phía trước thì giống nhau). Tương đương, từ các trạng thái của chúng, các trạng thái chấp nhận có thể chuyển tới cũng giống nhau. Do đó hai xâu này có bản chất như nhau trong suffix automaton và có thể gộp vào một trạng thái.

Tiếp theo xét hai xâu con A, B của S . Nếu $right_A$ và $right_B$ giao nhau, thì hiển nhiên một xâu là hậu tố của xâu còn lại; giả sử A là hậu tố của B , khi đó dễ thấy

$$right_A \supset right_B.$$

Nghĩa là, với hai xâu con bất kỳ của S , hai tập $right$ của chúng hoặc rời nhau, hoặc một tập chứa tập kia. Ta định nghĩa trạng thái cha của một trạng thái s , ký hiệu fa_s , là trạng thái thỏa

$$right_s \subset right_{fa_s}$$

và $|right_{fa_s}|$ nhỏ nhất. Khi đó mọi trạng thái tạo thành một cấu trúc cây, gọi là cây parent.

Rõ ràng tập $right$ của trạng thái đầu chứa mọi số nguyên từ 1 đến $|S|$. Với mỗi nút lá, kích thước tập $right$ của nó là 1. Do đó số nút lá là $O(|S|)$. Mỗi nút không phải lá có ít nhất hai con; nếu chỉ có một con thì có thể gộp nó với con đó. Vì thế số nút không phải lá nhiều nhất cũng là $O(|S|)$, và số trạng thái của suffix automaton là $O(|S|)$.

Xét các chuyển. Vì số trạng thái là $O(|S|)$, ta xét một đồ thị cây của automaton bắt đầu từ trạng thái đầu; số cạnh trên cây hiển nhiên là $O(|S|)$. Vậy chỉ cần xét các cạnh không thuộc cây.

Xét một cạnh không thuộc cây $u \rightarrow v$. Từ trạng thái đầu, đi theo cạnh cây tới u , rồi đi qua cạnh không thuộc cây này tới v , sau đó tiếp tục đi xuống thì chắc chắn tới được ít nhất một hậu tố. Nói cách khác, nếu mỗi hậu tố phủ cạnh không thuộc cây đầu tiên mà nó đi qua, thì mọi cạnh không thuộc cây đều được phủ. Vì vậy số cạnh không thuộc cây không vượt quá số hậu tố. Ta đã chứng minh được số chuyển của suffix automaton là $O(|S|)$.

Phương pháp xây dựng. Suffix automaton có một cách xây dựng rất đơn giản: phương pháp tăng dần.

Với mỗi trạng thái s , ghi len_s là độ dài xâu con dài nhất mà nó đại diện.

Giả sử hiện tại ta đã có suffix automaton của $|S| - 1$ ký tự đầu, và trong automaton hiện tại, xâu $S[1, |S| - 1]$ nằm ở trạng thái $last$. Bây giờ thêm ký tự thứ $|S|$, giả sử là c . Ta tạo trạng thái mới np ; rõ ràng $len_{np} = |S|$.

Sau đó xét các chuyển. Hiển nhiên ta cần thêm một chuyển $last \rightarrow np$, và cũng cần thêm một chuyển $fa_{last} \rightarrow np$, cứ như vậy cho đến khi gặp một trạng thái đã có chuyển bằng ký tự c . Giả sử trạng thái này là p , và trạng thái sau khi đi bằng c là q .

Lúc này $right_q$ sẽ có thêm một xâu: xâu có đầu mút phải là $|S|$ và độ dài dài nhất là $len_p + 1$. Có hai trường hợp:

1. $len_q = len_p + 1$. Khi đó mọi xâu do q đại diện vẫn có cùng tập $right$, nên chỉ cần đặt $fa_{np} = q$.
2. $len_q > len_p + 1$. Trong các xâu do q đại diện, các xâu có độ dài không quá $len_p + 1$ sẽ có thêm giá trị $|S|$ trong tập $right$, còn các xâu dài hơn thì không. Để duy trì tính chất mọi xâu trong một trạng thái có cùng tập $right$, ta tạo một trạng thái mới nq . Trạng thái nq đại diện cho mọi xâu do q đại diện trước đây có độ dài không quá $len_p + 1$, nên $len_{nq} = len_p + 1$; các thuộc tính khác của nq (fa và các chuyển) giống hệt q . Đồng thời dễ thấy $fa_q = fa_{np} = nq$.

Sau đó ta lại bắt đầu từ p . Trước đây chuyển bằng ký tự c của p tới q ; bây giờ nó phải chuyển tới nq . Tương tự, nếu chuyển bằng ký tự c của fa_p tới q , thì nó cũng phải chuyển tới nq , cứ như vậy cho đến khi gặp một điểm mà chuyển bằng ký tự c không tới q .

Đến đây, ta hoàn thành việc xây dựng suffix automaton.

6.1.3 Một vài tính chất cơ bản của suffix automaton

Các tính chất cơ bản đã được nhắc tới ở trên, tổng kết như sau:

Tính chất 1. Các độ dài của những xâu mà mỗi trạng thái s đại diện tạo thành khoảng

$$(\text{len}_{\text{fa}_s}, \text{len}_s].$$

Tính chất 2. Mọi xâu mà một trạng thái s đại diện có cùng số lần xuất hiện trong xâu gốc và cùng các vị trí đầu mút phải của mỗi lần xuất hiện.

Tính chất 3. Trong cây parent, tập right của mỗi trạng thái là tập con của tập right của trạng thái cha.

6.1.4 Cách tính tập right

Trước hết, thêm i vào tập right của trạng thái chứa pre_i .

Sau đó, theo thứ tự độ sâu giảm dần trong cây parent, lần lượt gộp tập right của mỗi trạng thái vào tập right của trạng thái cha.

Như vậy có thể tính được tập right của mỗi trạng thái, nhưng làm trực tiếp tốn $O(|S|^2)$.

Tùy bài toán cụ thể, ta có thể chỉ cần kích thước, giá trị lớn nhất, hay một số đại lượng khác của tập right; các đại lượng này có thể tính trong $O(|S|)$. Nếu thật sự cần tập right của từng trạng thái, có thể dùng cây cân bằng để duy trì, khi gộp dùng heuristic merging,² đạt độ phức tạp $O(|S| \log |S|)$. Nếu cần online, có thể dùng cây cân bằng persistent.

6.1.5 Dùng suffix automaton để tìm suffix tree và suffix array

Ngoài suffix automaton, suffix tree và suffix array cũng là các cấu trúc dữ liệu hậu tố thường dùng. Tuy nhiên việc xây dựng chúng thường không tiện: xây suffix tree tuyến tính có thể nói là khá rắc rối, còn xây suffix array tuyến tính có hằng số tương đối lớn. Vì suffix automaton có hằng số nhỏ và cách xây dựng khá ngắn gọn, ta có thể dùng nó để tìm suffix tree và suffix array.

Tính chất 4. Cây parent của suffix automaton chính là cây tiền tố ngược của xâu gốc.³

Trước hết, cây tiền tố ngược được định nghĩa như sau: đưa xâu đảo của mỗi tiền tố vào một trie, rồi gộp các dây không phân nhánh giống như trong suffix tree.

Tính chất này dễ nhận ra. Xét trạng thái của một tiền tố trên suffix automaton. Ta liên tục đi theo con trỏ fa; mỗi lần, xâu hiện tại trở thành một hậu tố của xâu trước đó, cho tới khi thành xâu rỗng. Nếu nhìn ngược quá trình này, đó chính là đi từ trên xuống dưới trong cây tiền tố ngược.

Vậy nếu ta xây suffix automaton của xâu đảo của xâu ban đầu, thì cây parent của nó chính là suffix tree của xâu ban đầu.

Để khôi phục suffix tree, ta có thể tìm một giá trị bất kỳ trong tập right của mỗi trạng thái trên suffix automaton, chẳng hạn giá trị lớn nhất. Sau đó dễ dàng xác định xâu nằm trên cạnh nối mỗi nút với cha của nó trong suffix tree.

²Vấn đề heuristic merging trên cây cân bằng không liên quan trực tiếp tới bài viết này nên không bàn thêm; độc giả quan tâm có thể xem lời giải bài *Cây non và tập hợp*.

³Thực chất chính là suffix tree của xâu đảo.

Cuối cùng, chỉ cần DFS suffix tree một lần là có thể thu được suffix array.

6.2 Ứng dụng của suffix automaton

6.2.1 Một cách diễn giải

Khi nói “đưa một xâu chạy trên suffix automaton”, ta có thể liên tưởng tới cách chạy KMP và AC automaton: bắt đầu từ trạng thái đầu; nếu tồn tại chuyển tương ứng thì đi theo chuyển đó, nếu không thì nhảy ngược theo con trỏ fa, cho tới khi tồn tại chuyển tương ứng hoặc đã nhảy ra khỏi suffix automaton.

6.2.2 Một vài bài toán kinh điển

Trước hết ta thảo luận lời giải của một vài bài toán kinh điển.

Xâu con chung dài nhất. Ta xây suffix automaton cho một trong hai xâu. Đưa xâu còn lại chạy trên suffix automaton và luôn duy trì độ dài xâu con hiện tại.

Với xâu con chung dài nhất của nhiều xâu (nhiều hơn 2), ta xây suffix automaton cho một xâu, rồi lần lượt đưa các xâu còn lại chạy trên automaton. Với mỗi trạng thái, duy trì độ dài khớp dài nhất của trạng thái đó đối với từng xâu. Sau đó, với mỗi trạng thái lấy min của các độ dài khớp dài nhất trên mọi xâu, rồi lấy max trên mọi trạng thái.

Xâu con thứ k theo thứ tự từ điển. Ta topo một lượt suffix automaton để tính từ mỗi điểm có thể đi tới bao nhiêu xâu con. Sau đó DFS từ trạng thái đầu: nếu số xâu con mà trạng thái hiện tại có thể đi tới không nhỏ hơn k thì tiếp tục DFS; nếu không, trừ k đi số xâu con đó rồi quay lui.

Nếu cần tìm xâu con thứ k khác nhau thì không cần kích thước tập right; nếu tính cả số lần xuất hiện thì cần kích thước right của mỗi điểm.

Hậu tố nhỏ nhất theo thứ tự từ điển (biểu diễn vòng nhỏ nhất). Sao chép xâu gốc một lần và nối vào phía sau, rồi xây suffix automaton.

Bắt đầu từ trạng thái đầu, mỗi lần đi theo chuyển nhỏ nhất theo thứ tự từ điển; sau khi đi $|S|$ lần, xâu nhận được chính là biểu diễn vòng nhỏ nhất.

Nếu cần hậu tố nhỏ nhất, ta thêm vào sau xâu gốc một ký tự kết thúc nhỏ hơn mọi ký tự trong bảng chữ cái, rồi sao chép xâu một lần nữa là được.

Tiếp theo ta dùng một số ví dụ đơn giản để thảo luận các ứng dụng cơ bản của suffix automaton.

Ví dụ 1: Strings. Nguồn bài: Adera 1 Cup, bài mô phỏng trại đông.

Tóm tắt đề. Cho n xâu, hỏi với mỗi xâu có bao nhiêu xâu con không rỗng là xâu con của ít nhất k trong n xâu.

$$1 \leq n, k \leq 10^5, \quad \sum |S_i| \leq 10^5.$$

Phân tích. Trước hết nối n xâu lại với nhau, giữa các xâu dùng một ký tự không nằm trong bảng chữ cái gốc để phân tách. Sau đó xây suffix automaton của xâu nối này. Với mỗi nút trong suffix automaton, ta cần tính nó là xâu con của bao nhiêu trong n xâu ban đầu.

Ý tưởng cơ bản là đưa từng xâu chạy trên suffix automaton. Với mỗi nút đi tới, tăng số lần xuất hiện của mọi nút trên đường đi theo con trỏ fa lên gốc thêm 1. Nhưng làm vậy sẽ gây trùng lặp. Ta có thể ghi lại ở mỗi trạng thái xâu cuối cùng đã đi tới nó; khi tới một điểm, chỉ cần đi ngược lên tới điểm đầu

tiên mà xuất hiện tại đã từng đi tới là đủ. Nếu làm thô bạo, độ phức tạp là $O(n\sqrt{n})$.⁴ Dùng heavy-light decomposition có thể đạt $O(n \log^2 n)$.

Sau đó chỉ cần quy hoạch một lượt để tính trên đường từ mỗi nút theo fa lên gốc có bao nhiêu xâu xuất hiện ít nhất k lần. Cuối cùng đưa từng xâu chạy trên suffix automaton, cộng giá trị của mỗi nút mà nó đi tới là được.

Ví dụ 2: Palindrome. *Nguồn bài: APIO 2014.*

Tóm tắt đề. Cho một xâu độ dài n , tìm giá trị lớn nhất của số lần xuất hiện nhân với độ dài trên mọi xâu con palindrome của nó.

$$1 \leq n \leq 3 \times 10^5.$$

Phân tích. Xây suffix automaton của xâu gốc và tính giá trị lớn nhất rmax trong tập right của mỗi nút.

Sau đó đưa xâu đảo chạy trên suffix automaton. Nếu xâu khớp hiện tại trong xâu đảo, ký hiệu $[l, r]$, phủ rmax của nút hiện tại, thì $[l, rmax]$ là một palindrome.⁵

Chi tiết xem tài liệu tham khảo [4].

Ví dụ 3: Nhận diện xâu con. *Tóm tắt đề.* Cho một xâu độ dài n . Định nghĩa xâu con nhận diện của vị trí i là một xâu con chứa vị trí này và chỉ xuất hiện một lần trong xâu gốc. Hãy tìm độ dài xâu con nhận diện ngắn nhất cho mỗi vị trí.

$$1 \leq n \leq 10^5.$$

Phân tích. Hiển nhiên mọi xâu con nhận diện đều nằm ở những nút có kích thước tập right bằng 1 trong suffix automaton. Vì thế trước hết ta tìm tất cả các nút như vậy. Với mỗi nút có kích thước tập bằng 1, ta dễ dàng biết đầu mút phải duy nhất của lần xuất hiện.

Nếu một xâu xuất hiện đúng một lần, thì mở rộng nó sang trái thêm một đoạn vẫn xuất hiện đúng một lần. Do đó, với mỗi nút có kích thước tập right bằng 1, ta tính đầu mút phải này cần mở rộng sang trái ít nhất bao xa để vẫn có số lần xuất hiện bằng 1. Khi đó mỗi tập right tương ứng với một đoạn và một giá trị lấy min, cùng với một tiền tố và một cấp số cộng có công sai -1 lấy min. Việc này có thể thực hiện bằng cách sắp xếp rồi mô phỏng đơn giản.

Độ phức tạp là $O(n \log n)$.

Ví dụ 4: Difference. *Nguồn bài: AHOI 2013.*

Tóm tắt đề. Cho một xâu độ dài n , tính tổng độ dài LCP của mọi cặp hậu tố của nó.

$$1 \leq n \leq 5 \times 10^5.$$

Phân tích. Trước hết đảo xâu này lại. Khi đó bài toán trở thành tính tổng độ dài hậu tố chung dài nhất của mọi cặp tiền tố.⁶

Tính chất 5. Hậu tố chung dài nhất của hai xâu nằm ở trạng thái LCA của hai trạng thái tương ứng với hai xâu đó trên cây parent.

Lý do là mọi tổ tiên của trạng thái tương ứng với một xâu đều là hậu tố của nó, và độ sâu càng lớn thì độ dài càng lớn.

⁴Phải có xâu rất đặc biệt mới đạt tới độ phức tạp này, và hằng số rất nhỏ.

⁵Ở đây chỉ vị trí trong xâu gốc.

⁶Ở đây đã trình bày đề bài sau khi biến đổi đơn giản từ đề gốc.

Vì vậy, ta tô đen các nút chứa từng tiền tố. Bài toán trở thành: mỗi nút là LCA của bao nhiêu cặp nút đen. Đây là một bài toán đơn giản; chỉ cần quy hoạch từ dưới lên trên trong cây parent.

Suffix automaton cũng có thể kết hợp với các cấu trúc dữ liệu và công cụ xử lý xâu khác để giải một số bài toán khó hơn.

Ví dụ 5: Jeweler. Nguồn bài: CTSC 2010.

Tóm tắt đề. Cho một cây gồm n đỉnh, mỗi đỉnh có một ký tự. Cho thêm một xâu S độ dài m . Với mỗi cặp đỉnh, các đỉnh trên đường đi giữa chúng tạo thành một xâu; tính tổng số lần xuất hiện trong S của tất cả các xâu như vậy.

$$1 \leq n, m \leq 5 \times 10^4.$$

Phân tích. Xét dùng centroid decomposition. Vì n, m cùng bậc, khi tính độ phức tạp ta thống nhất dùng n .

1. Nếu kích thước khối phân rã nhỏ hơn $\sqrt{n}$, ta DFS trực tiếp mọi đường đi và thống kê số lần xuất hiện trong suffix automaton của S . Vì các khối này rời nhau, chi phí tính một khối là $O(\text{size}^2)$, nên tổng độ phức tạp nhiều nhất là $O(n\sqrt{n})$.
2. Nếu kích thước khối phân rã lớn hơn $\sqrt{n}$, xét cách tính các đường đi qua trọng tâm của khối hiện tại; các đường đi khác được xử lý đệ quy bằng phân rã.

Xét các vị trí xuất hiện trong S của ký tự c ở trọng tâm x . Nếu một đường đi có dạng $a \rightarrow x \rightarrow b$, thì tập right của đoạn $a \rightarrow x$ trong S chắc chắn là tập con của các vị trí xuất hiện của c trong S . Tương tự, nếu đảo đoạn $x \rightarrow b$, thì tập right của nó trong xâu đảo của S cũng chắc chắn là tập con của các vị trí xuất hiện của c trong S .

Vậy ta bắt đầu DFS khối phân rã từ x . Mỗi lần, duy trì vị trí của xâu hiện tại trên suffix tree của xâu gốc và của xâu đảo. Sau đó ta đẩy thông tin trên hai suffix tree một lượt từ trên xuống dưới để tính số lượng khớp của mỗi hậu tố, tức mỗi vị trí. Cuối cùng nhân số lượng khớp ở hai vị trí tương ứng trên hai suffix tree, ta thu được tổng số lần xuất hiện trong xâu gốc của các đường đi qua trọng tâm.

Chú ý cách làm này đếm thừa trường hợp a, b nằm trong cùng một cây con của trọng tâm; vì vậy với từng cây con của trọng tâm, ta dùng cùng phương pháp tính riêng một lượt rồi trừ đi.

Chi phí tính một khối phân rã là $O(\text{size} + n)$. Số khối phân rã có kích thước lớn hơn $\sqrt{n}$ nhiều nhất là $O(\sqrt{n})$,⁷ nên độ phức tạp là $O(n\sqrt{n})$.

Tóm lại, ta thu được một thuật toán tốt với độ phức tạp $O(n\sqrt{n})$.

Ví dụ 6: str. Nguồn bài: Feyat Cup 1.5; tác giả đề: Huang Zhixiang.

Tóm tắt đề. Cho hai xâu có độ dài lần lượt là n, m . Định nghĩa hai xâu là khớp nếu chúng có nhiều nhất một vị trí khác nhau. Hãy tìm xâu con chung dài nhất của hai xâu theo định nghĩa khớp này.

$$1 \leq n, m \leq 10^5.$$

Phân tích. Xét xâu cuối cùng có thể khớp được: ở giữa nó có một vị trí không khớp, còn hai phía trái và phải của vị trí đó khớp tương ứng. Nếu đã biết xâu phía trái, ta dễ biết mọi vị trí xuất hiện của nó trong hai xâu, rồi lấy LCP⁸ của các xâu phía phải tương ứng theo từng cặp là được.

Ta nối hai xâu lại, ở giữa thêm một ký tự không thuộc bảng chữ cái gốc, rồi xây suffix automaton của xâu này. Xét một nút trong suffix automaton: trong tập right của nó, một phần xuất hiện bên trong xâu

⁷Ý tưởng chứng minh: sau mỗi lần phân rã, kích thước cây con giảm ít nhất một nửa; sau k lần phân rã, khối lớn nhất không vượt quá $n/2^k$ và số khối là 2^k . Đặt khối lớn nhất không vượt quá $\sqrt{n}$ là thu được kết luận.

⁸Longest common prefix.

thứ nhất, phần còn lại xuất hiện bên trong xâu thứ hai. Ví dụ có một giá trị right là a , nằm trong xâu thứ nhất, và một giá trị right khác là b , nằm trong xâu thứ hai. Khi đó LCP của $a + 2$ và $b + 2$ chính là độ dài khớp dài nhất tương ứng. Nếu ta tính được LCP tương ứng của mọi cặp (a, b) như vậy, lấy giá trị lớn nhất cộng với len của nút hiện tại, thì đó là đáp án của nút này. Lấy lớn nhất trên mọi nút là đáp án cuối cùng.

Tuy nhiên tính LCP cho mọi cặp (a, b) chắc chắn không đủ nhanh. Ta xét thứ tự mọi hậu tố. Với một nút, nếu trộn mọi a và b rồi sắp theo thứ tự hậu tố tương ứng, thì ta chỉ cần tính LCP của các cặp kề nhau sau khi sắp xếp mà xuất hiện trong hai xâu khác nhau. Với bài toán này, ta xét các nút theo thứ tự từ dưới lên trên trên cây parent, dùng cây cân bằng để duy trì toàn bộ dãy. Khi gộp cây cân bằng, duy trì thêm thông tin cần thiết, ta đạt độ phức tạp

$$O((n + m) \log(n + m)).$$

6.3 Suffix automaton của cây chữ cái

Ta có thể mở rộng thuật toán KMP lên cây chữ cái để tạo thành AC automaton. Tương tự, ta cũng có thể mở rộng suffix automaton lên cây chữ cái.

Trước hết xem một ví dụ.

Ví dụ 7: Xâu của pty. Tác giả đề: Peng Tianyi.

Tóm tắt đề. Cho một cây có gốc gồm n nút, mỗi cạnh có một ký tự. Định nghĩa một đường đi là bắt đầu từ một nút nào đó, đi xuống tới một nút nào đó; các ký tự trên cạnh tạo thành một xâu. Hai xâu khớp nếu chúng hoàn toàn giống nhau. Cho thêm một xâu độ dài m , hỏi có bao nhiêu cặp khớp giữa xâu con của nó và đường đi trên cây.

$$1 \leq n \leq 8 \times 10^5, \quad 1 \leq m \leq 8 \times 10^6.$$

Phân tích. Cách cơ bản là xây suffix automaton cho xâu và tính kích thước tập right, rồi DFS cây chữ cái.

Nhưng trong bài này, vì xâu quá dài, xây suffix automaton cho xâu không chấp nhận được về bộ nhớ.⁹ Cần tìm hướng khác.

Xét xây suffix automaton cho cây chữ cái đã cho. Khi chèn một nút, ta dùng trạng thái của tiền tố của cha nó trong suffix automaton làm trạng thái last để chèn.¹⁰ Nhưng có một vấn đề: một nút có thể có hai con mà ký tự trên cạnh nối tới chúng giống nhau.

Cách xử lý thực ra rất đơn giản: gộp các nút con có cùng ký tự dưới cùng một nút; kích thước right ban đầu của nút sau khi gộp là số nút được gộp.

Cuối cùng, đưa xâu đã cho chạy trên suffix automaton và thống kê đáp án.

Xem thêm một ví dụ khác.

Ví dụ 8: Vùng đất huyền tường được chư thần ưu ái. Nguồn bài: ZJOI 2015; tác giả đề: Chen Lijie.

Tóm tắt đề. Cho một cây gồm n nút, mỗi nút có một ký tự. Định nghĩa một đường đi là xâu tạo bởi các ký tự trên các nút giữa hai điểm nào đó. Hỏi trên cây có bao nhiêu đường đi đôi một khác nhau.

$$1 \leq n \leq 10^5,$$

bảng chữ cái có kích thước 10, và cây có nhiều nhất 20 nút lá.

⁹Có lẽ cách lưu cạnh bằng hash vẫn dùng được.

¹⁰Tiền tố ở đây là đường đi từ gốc tới nút đó.

Phân tích. Chú ý đặc điểm của đề: số nút lá rất ít. Vì vậy ta lần lượt lấy mỗi nút lá làm gốc, rồi hợp nhất các cây thu được để tạo thành một cây chữ cái.

Bài toán tiếp theo là trên cây chữ cái này có bao nhiêu đường đi từ trên xuống dưới khác nhau. Ta xây suffix automaton cho nó; với mỗi nút, cộng

$$\text{len} - \text{len}_{\text{fa}}$$

vào đáp án.

6.4 Tổng kết

Là một công cụ xử lý xâu mới nổi trong OI, suffix automaton có ưu điểm là chức năng toàn diện, mã nguồn ngắn gọn, độ phức tạp thấp và hằng số nhỏ. Tương ứng, trong quá trình ứng dụng, nó đòi hỏi ta phải suy nghĩ và nghiên cứu nhiều hơn để hiểu được vẻ đẹp của suffix automaton.

6.5 Lời cảm ơn

- Xin cảm ơn CCF đã cung cấp nền tảng học tập và giao lưu.
- Xin cảm ơn Chen Lijie đã giới thiệu suffix automaton.
- Xin cảm ơn bạn Shi Wenbin đã dạy tôi suffix automaton.
- Xin cảm ơn các bạn Chen Xiaobo và Lu Kaifeng đã giúp đỡ trong quá trình viết luận văn.
- Xin cảm ơn thầy huấn luyện Zhou Zusong đã hỗ trợ.

Tài liệu tham khảo của bài viết

- [1] Chen Lijie, *Suffix Automaton*, trao đổi học viên trại đông 2012.
- [2] Fan Haoqiang, *Suffix automaton và xây dựng suffix tree tuyến tính*.
- [3] WRIA, *Lời giải mới cho CTSC 2010 Jeweler*, bài tập đội tuyển quốc gia 2013.
- [4] Zhang Tianyang, *Lời giải APIO 2014 Palindrome*, bài tập đội tuyển quốc gia 2015.
- [5] Chen Lijie, *Lời giải ZJOI 2015 Day 1*.
- [6] #22334, *Lời giải Feyat Cup 1.5*.